

ISP器件的设计与应用

——组合逻辑电路设计(HDL)并仿真下载实现

BASYS2 实验开发板

不能带出实验室

ISP器件的设计与应用

ISP(In-System Programming):在系统可编程

传统的PLD(programmable logic device)可编程逻辑器件: “**先编程后装配**”,需要**12V**编程信号, 并**必须**从电路板上**取下**, 使用**专用硬件**设备写入

- ◆ ISP PLD器件: “**先装配后编程**”, 使用**5V**或**3.3V**编程信号, 使得器件在焊接于电路板上的状态下即可进行编程或反复编程, 无需拆卸
- ◆ ISP PLD器件: 内部固化的Bootloader程序, 通过SPI、I2C、JTAG、UART、USB、CAN等通信接口、协议接收来自上位机的指令和数据, 允许开发者直接**在电路板**上对闪存(FLASH)或EEPROM进行编程
- ◆ ISP 被广泛应用于 MCU (如 STM32、AT89S52、STC 系列)、**FPGA** (如 Altera、**Xilinx**) 等

ISP器件的设计与应用

一 实验目的

二 实验原理

三 ISE开发全流程---流水灯为例

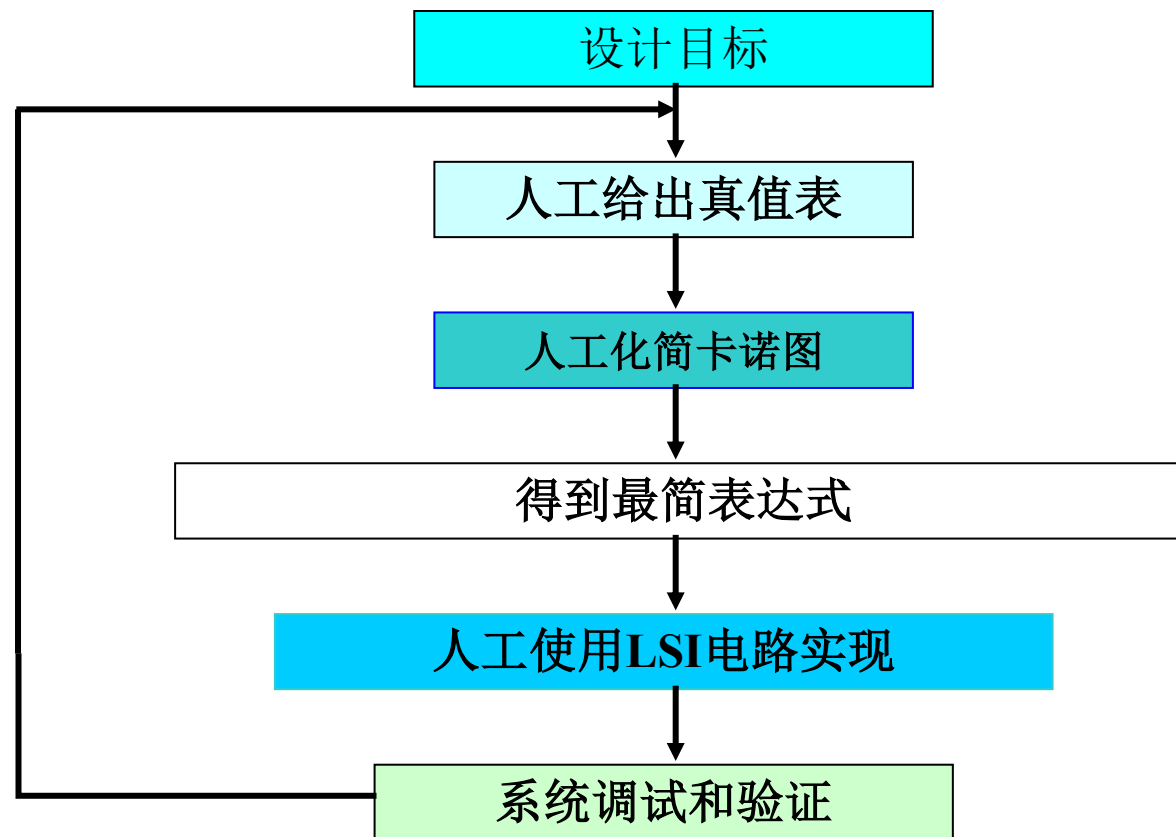
四 实验内容与要求

最后附有：常见问题

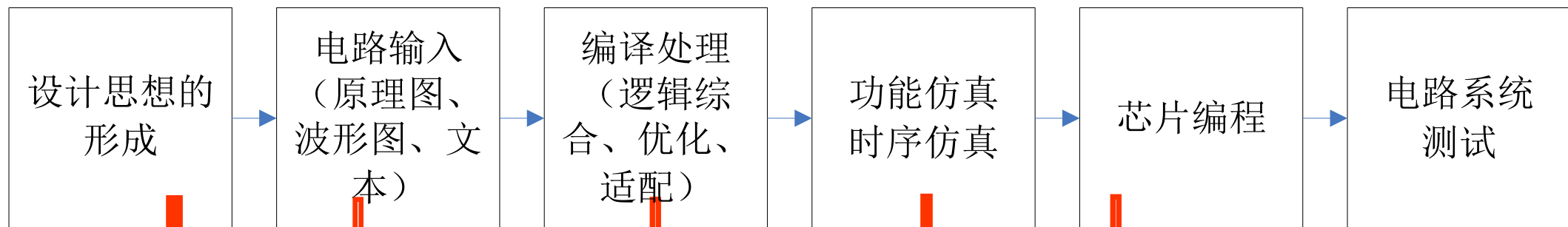
一 实验目的

- ◆ 可编程逻辑器件实现数字电路方法步骤
- ◆ Xilinx_ISE软件开发可编程器件
- ◆ verilog HDL描述数字逻辑电路
- ◆ 分层次、分模块的电路设计方法

一 实验目的 —— 传统数字系统设计流程



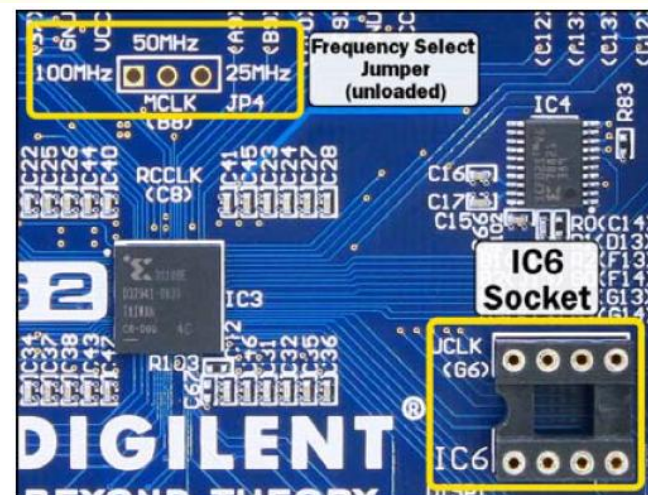
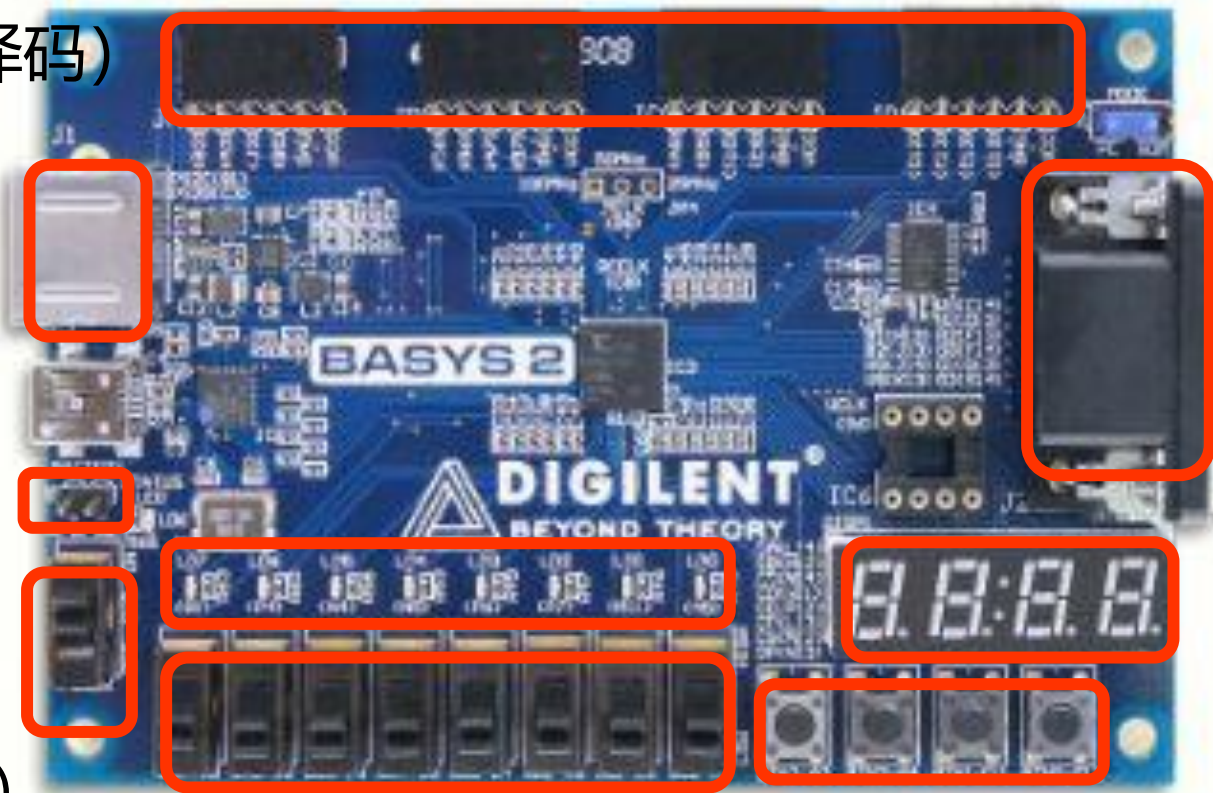
一 实验目的 —— 现代数字系统设计流程



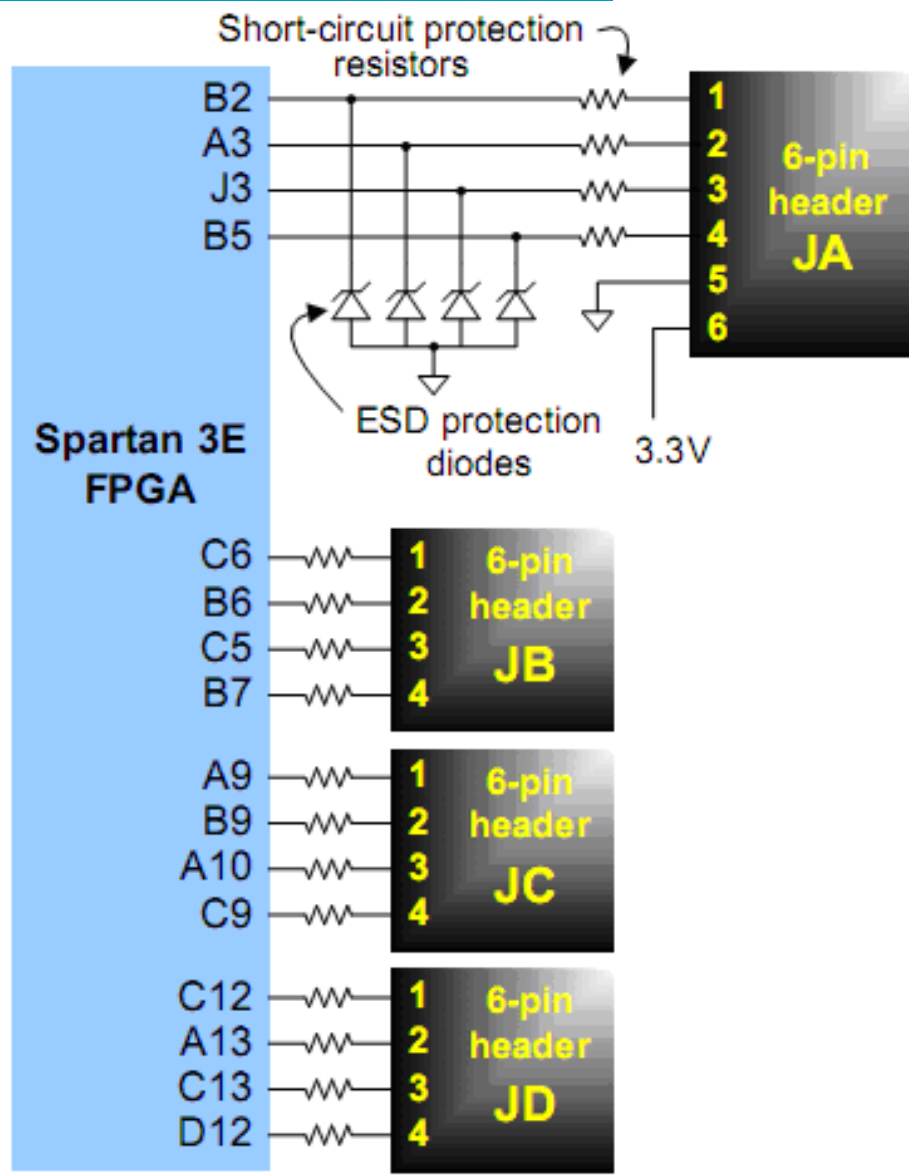
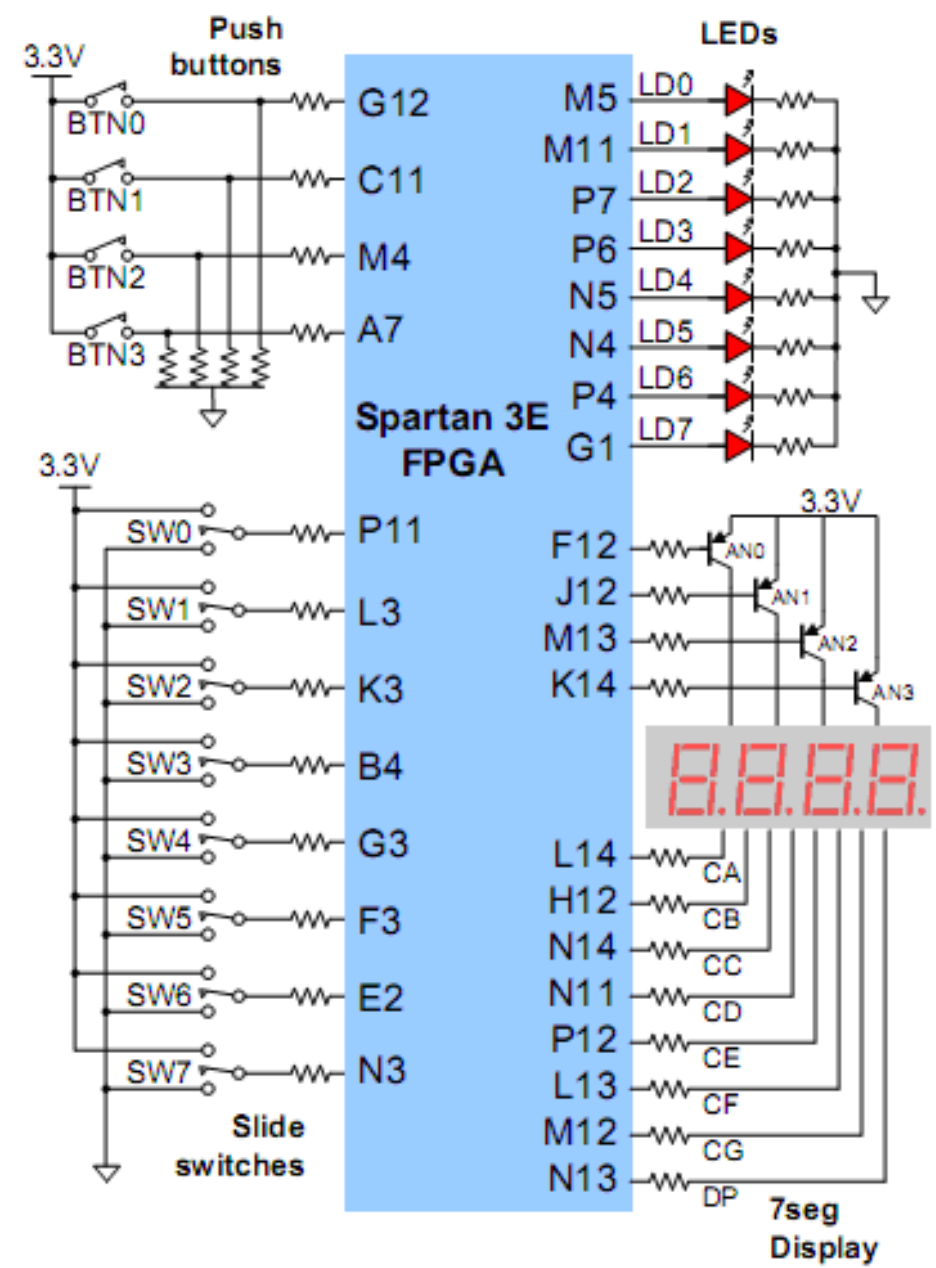
- ◆ 数字电路设计基本知识与设计思路：需求，输入输出等
- ◆ HDL语言：培养良好的风格和层次结构、纠正语法错误
- ◆ 保证实际语句是可综合的，占用的各种资源不超过硬件提供
- ◆ 测试模板TestBench：分析逻辑错误；分析时序要求
- ◆ 熟悉硬件平台，按照DataSheet资料正确分配资源和管脚号
- ◆ 合理设计外围电路，恰当的数据对接处理

二 实验原理 ——BASYS2实验板

- ◆4个七段数码管(AN3-AN0)(不含8421译码)
- ◆8个LED发光管显示(LD7-LD0)
- ◆4个按键开关 (BTN3-BTN0)
- ◆8个拨码开关 (SW7-SW0)
- ◆1个8位VGA显示接口
- ◆4个6针PMOD用户扩展接口
- ◆1个PS/2接口
- ◆USB2.0接口(供电/程序下载/数据通信)
- ◆可配置晶振(25MHz/50MHz/100MHz)
- ◆SW8拨码开关
拨到位置ON的时候电源打开，拨到位置OFF的时候电源关闭
- ◆电池接口直接供电 (两根插针的接口形式，供电电压3.5V~5.5V)

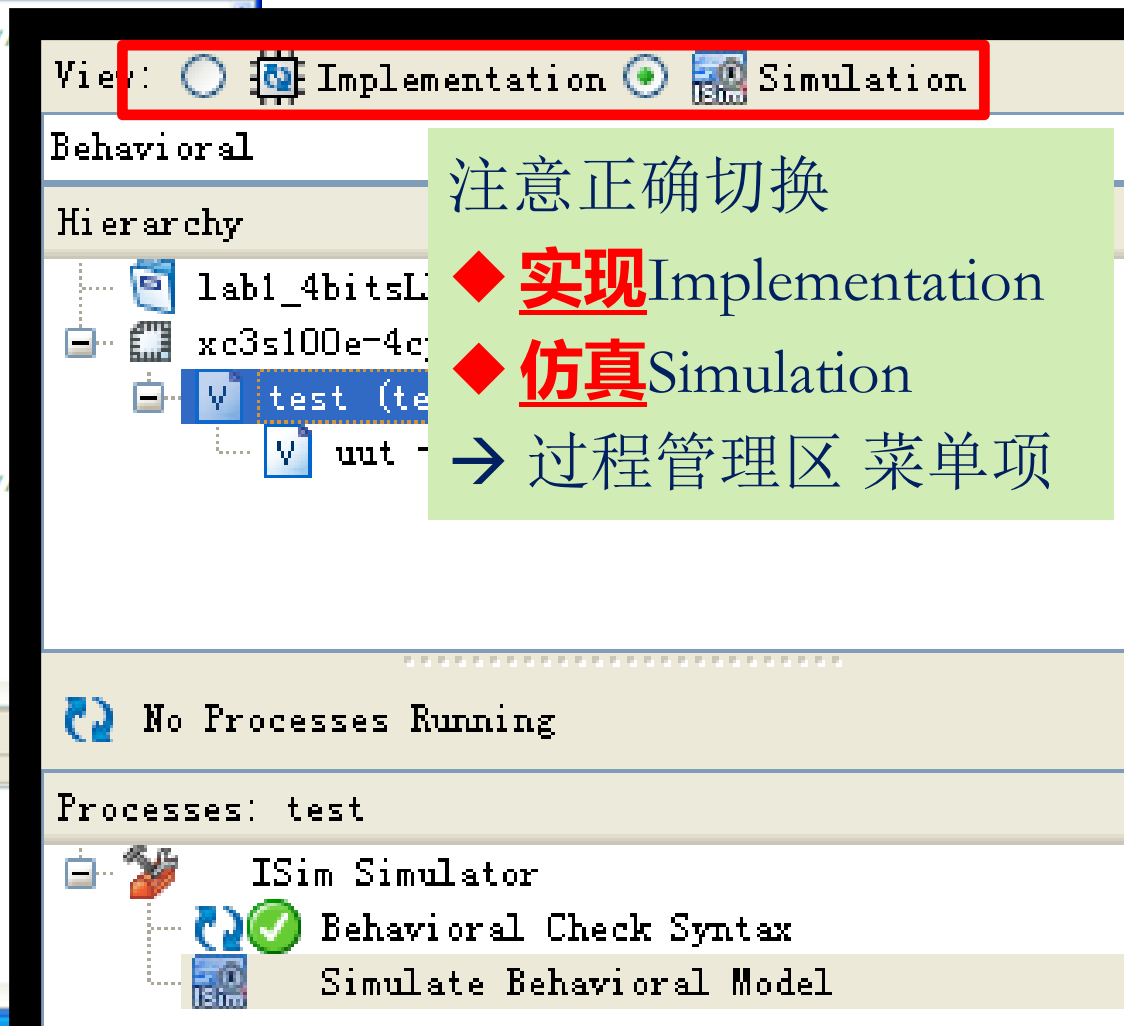
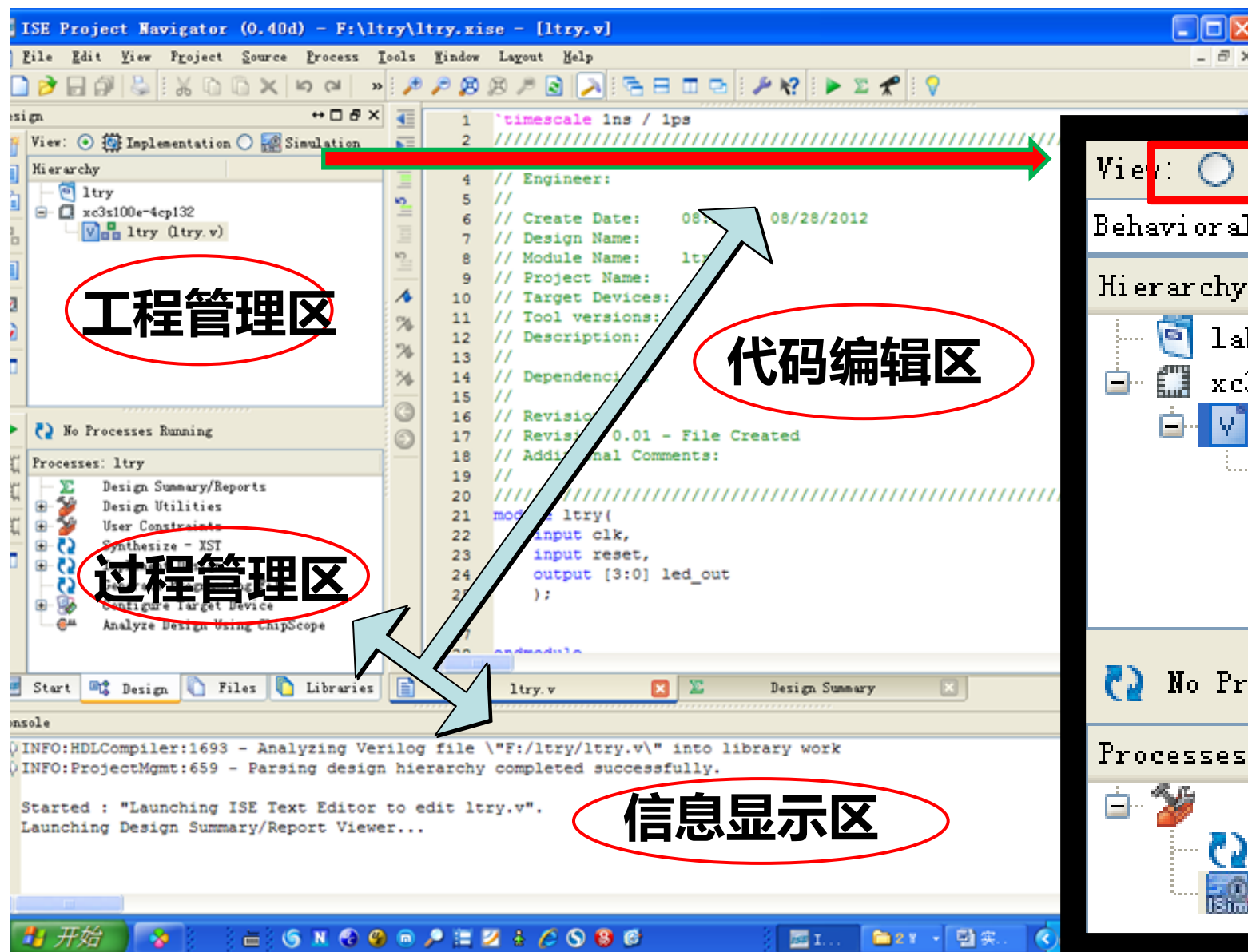


二 实验原理——BASYS2各IO管脚定义



二 实验原理 —— Xilinx开发环境—— ISE 14.x

开启ISE软件： Xilinx ISE Design Suite→ ISE Design Tools→ Project Navigator



三 ISE开发全流程---流水灯为例

- 1 新建工程Project**
- 2 在Project中创建新的设计文件**
- 3 编译检错并查看设计**
- 4 设计仿真**
- 5 创建硬件约束**
- 6 综合编译下载**

1 新建工程

(1) 开启ISE14.7软件



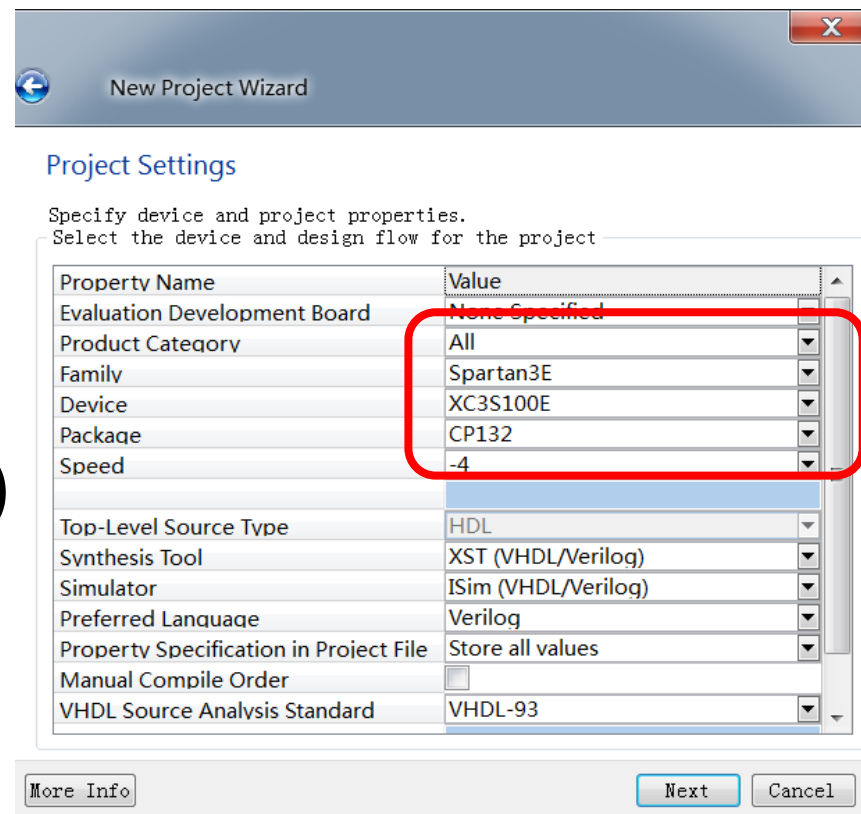
(2) ISE14开启工程: File→New Project

全英文目录和文件名

(3) 硬件FPGA参数设置

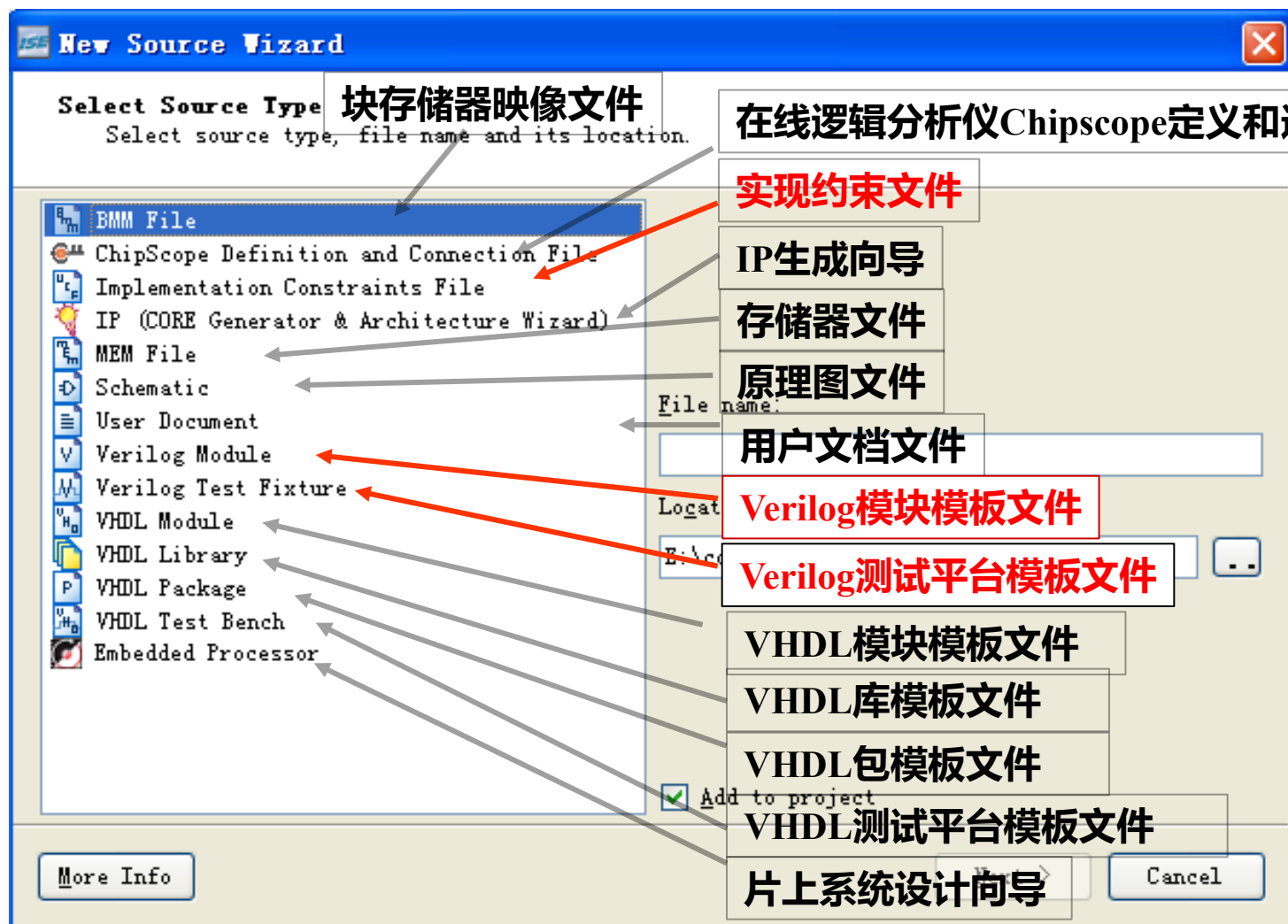
Basys2板

- Family(Spartan3E)/Device(XC3S100E)
- package(CP132)/Speed(-4)



2 在Project中创建新的Verilog文件

(1) Project → New Source; Verilog Module: 扩展名为.V



可以参考ISE所附的语言模板 Language Template(Edit菜单). 本实验计数器模块, 点击软件界面上方的语言模板的快捷键  然后选择 "Verilog -> Synthesis Constructs -> Coding Examples->Counters" 选择所需的计数器类型以做参考

流水灯(时序逻辑)——Verilog Module源码

```
module led(  
    input clk,  
    input reset,  
    output [3:0] led_out  
);  
reg [26:0] counter;
```

```
always @(posedge clk)  
    begin  
        if (reset)  
            counter <= 0;  
        else  
            counter <= counter + 1;  
    end
```

```
//      assign led_out = counter[3:0];
```

//仿真时将counter的低4位传给led_out以提高频率，加快仿真速度，看仿真波形

```
      assign led_out = counter[26:23];
```

//仿真完后下载到硬件前改为将counter的高4位传给led_out以便看清led闪烁

```
endmodule
```

◆ **always**内被赋值信号必需是**reg**型

◆ **(=)**:阻塞赋值,组合逻辑,顺序依次执行

◆ **(<=)**:非阻塞赋值,时序逻辑,同时并行执行

3 编译检错并查看电路

(1) 句法检查: 点开Synthesize-xst, 双击Check Syntax

(2) 双击Synthesize – xst进行综合

◆ 双击View RTL Schematic: 查看设计 RTL(Register Transfer Level,寄存器传输级) 电路模块

◆ 直接双击顶层电路, 则可以看到设计的内部 门级电路是由哪些资源组成的

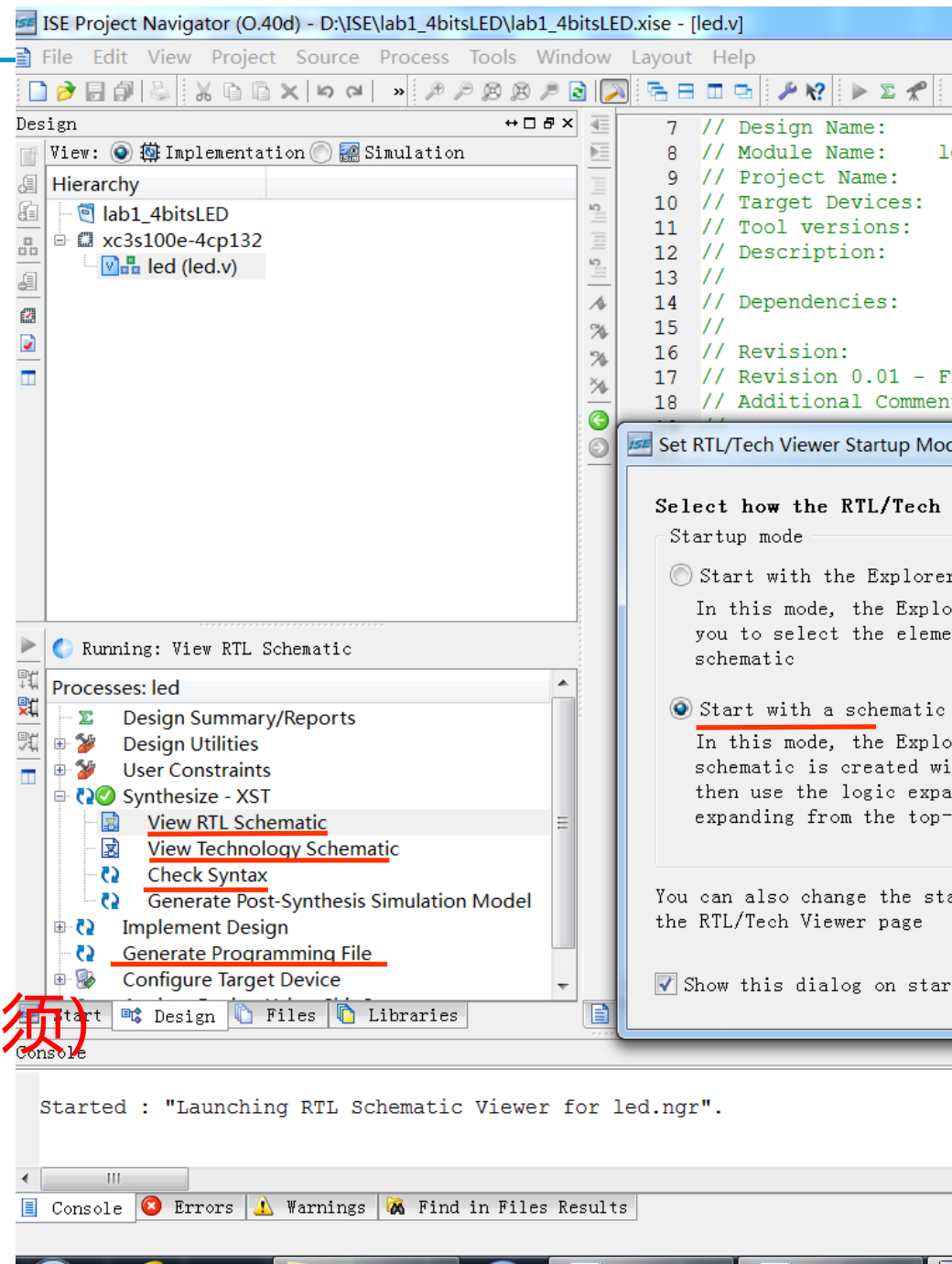
◆ 每一个运算映射到芯片底层资源, 独享

(3) 生成bit: 双击Generate Programming File

生成bit文件, 下载到开发板前完成下面任务

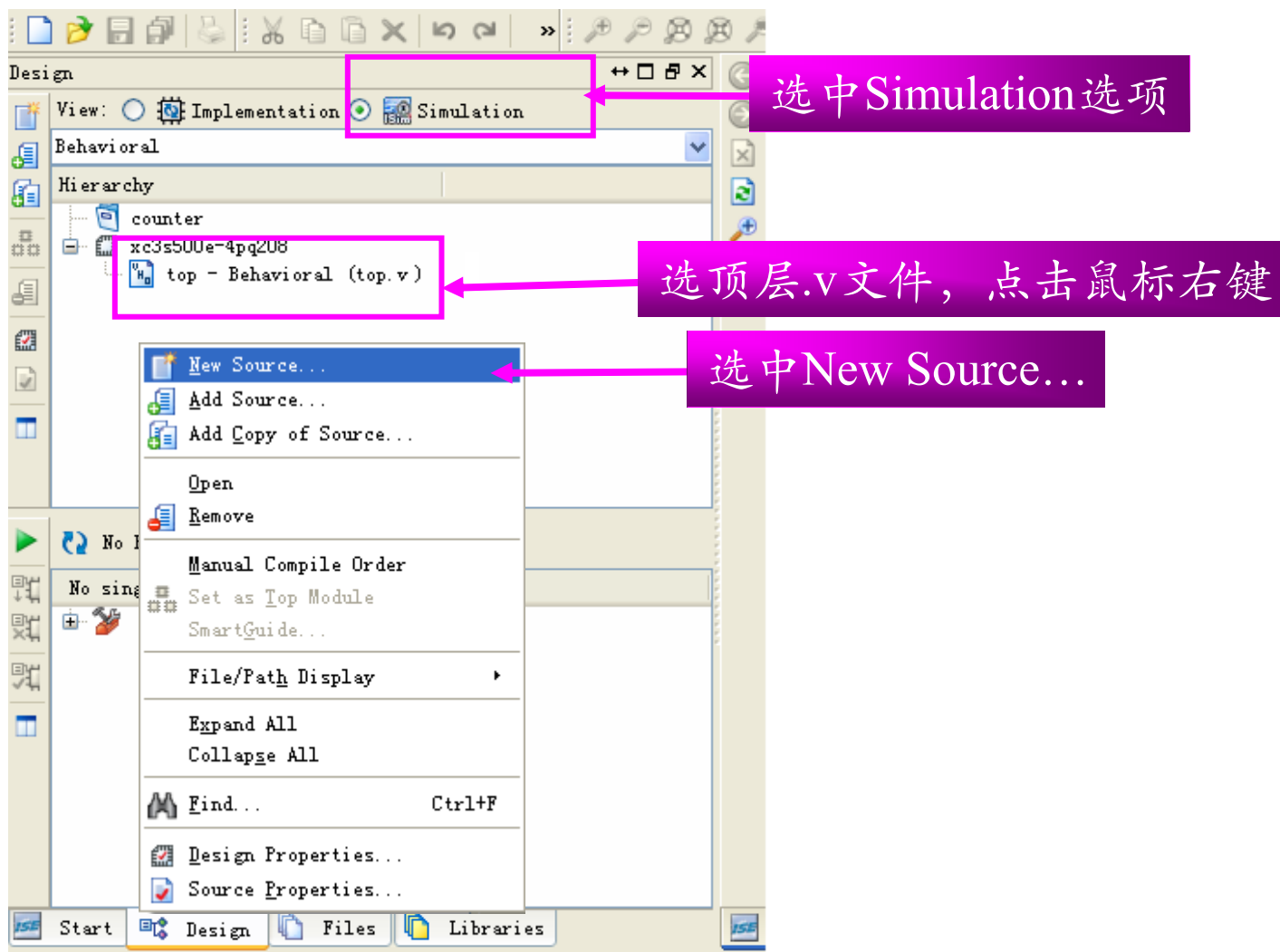
◆ 测试模板(Testbench)验证设计正确性(非必须)

◆ 约束创建 (必须)



4 设计仿真 ——验证代码的行为功能是否正确

(1) 点击led HDL文件,创建测试平台源文件**Verilog Test Fixture**



4 设计仿真 ——验证代码的行为功能是否正确

应**熟练掌握**HDL语句编写简单测试模块**TestBench**及相关仿真操作，能根据语句画波形！

主要有：

- 输入输出写法；
- 实例化语句；
- initial块 (并行)；
- #n延迟语句；
- timescale ；
- parameter；

```
24
25 module test;
26
27     // Inputs
28     reg clk;
29     reg reset;
30
31     // Outputs
32     wire [3:0] led_out;
33
34     parameter PERIOD = 10; //设定时钟周期为10ns
35
36     // Instantiate the Unit Under Test (UUT)
37     led uut (
38         .clk(clk),
39         .reset(reset),
40         .led_out(led_out)
41     );
42
43     //生成时钟
44     always begin
45         clk = 1'b0;
46         #(PERIOD/2) clk= 1'b1;
47         #(PERIOD/2);
48     end
49
50     initial begin
51         // Initialize Inputs
52         clk = 1'b0;
53         reset = 1;
54
55         // Wait 500 ns for global reset to finish
56         #500;|
57
58         // Add stimulus here
59         reset = 0;
60
61     end
62
63 endmodule
64
```

编辑输入激励：

- 时钟周期10ns
- 复位信号为高持续500ns
- 再将复位信号置低

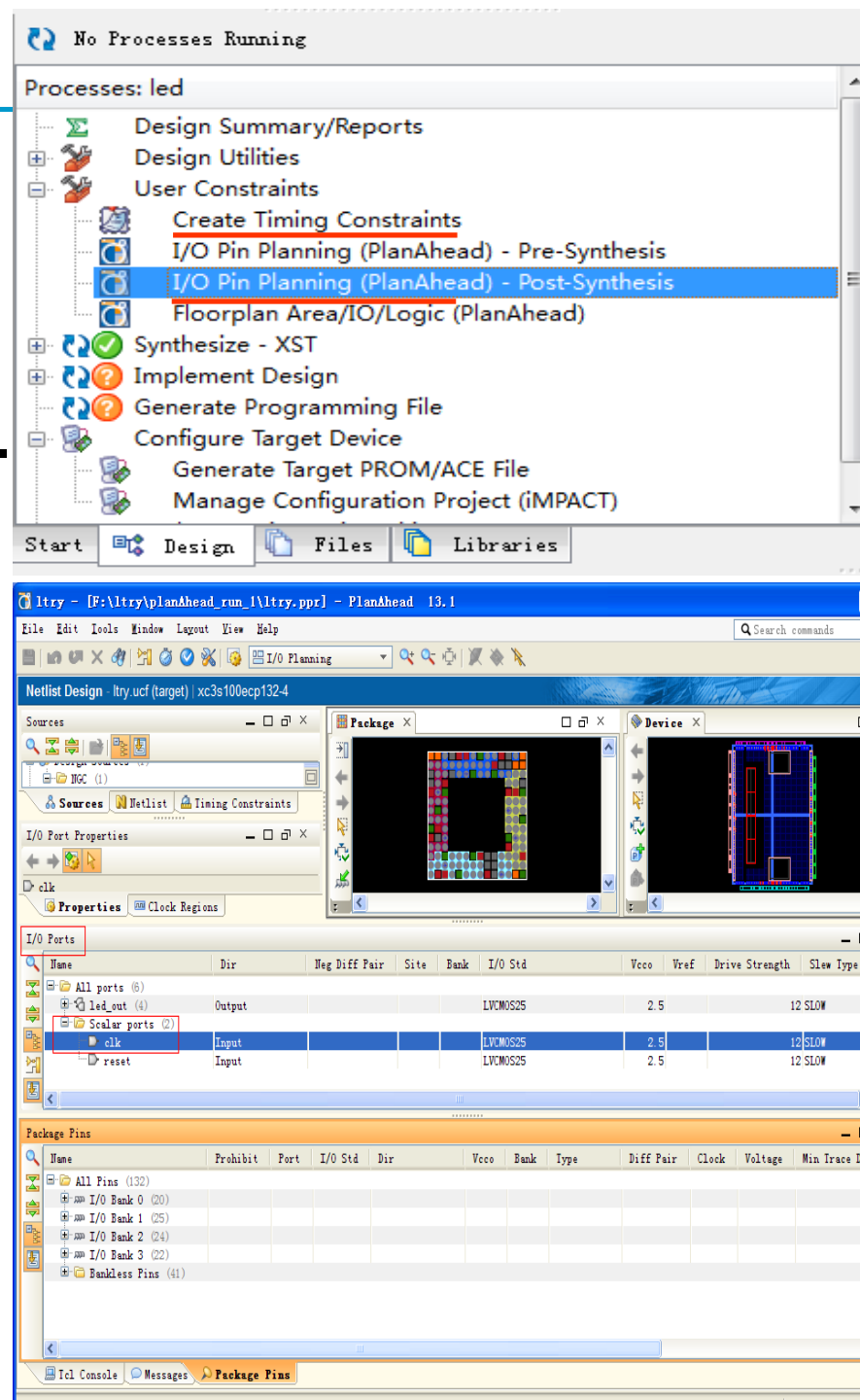
5 创建硬件约束

约束主要包括全局时序约束（周期约束）和管脚约束

- 1) 图形化界面设置：软件自动生成约束
(自动生成UCF文件)
- 2) 直接编辑UCF文本文件 **Project → New Source.**
文件格式为**Implementation Constraints File**
- 3) 两种方法相结合

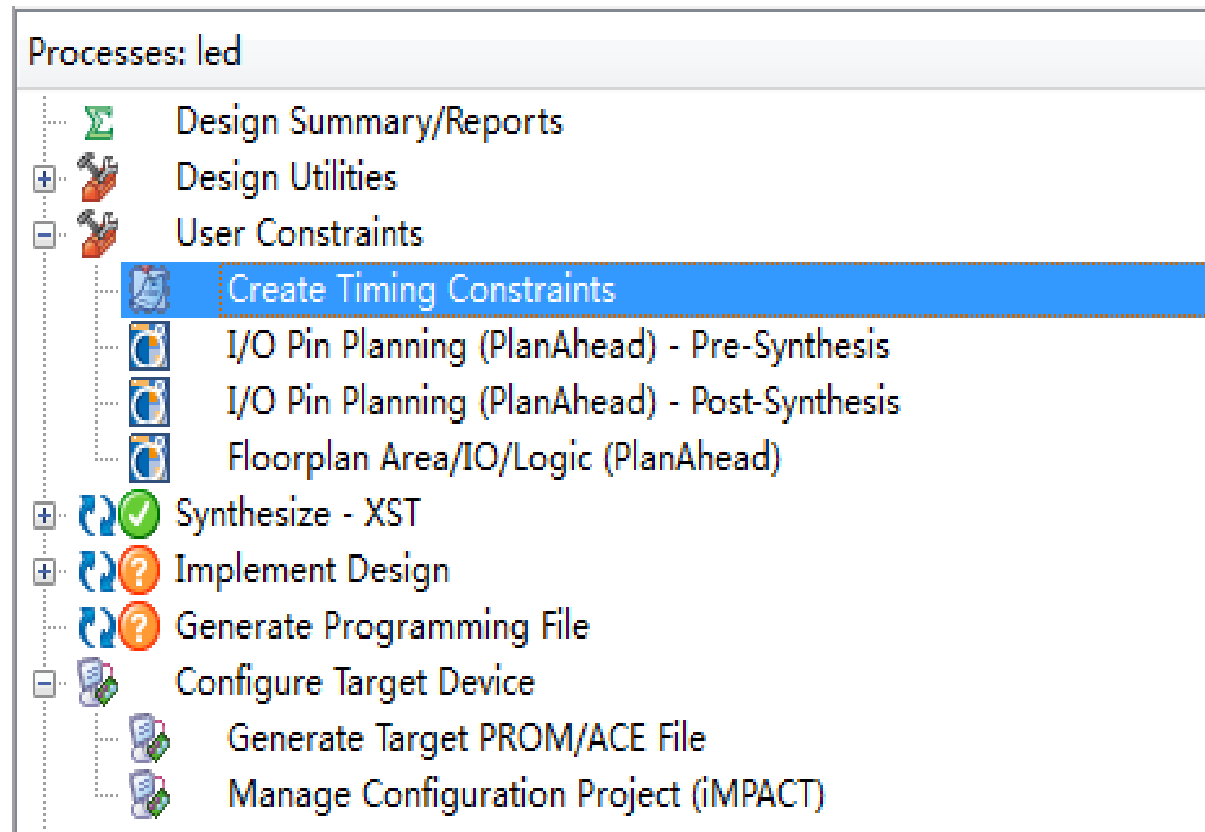
```
1 NET "clk" LOC = B8;
2 NET "led_out[0]" LOC = M5;
3 NET "led_out[1]" LOC = M11;
4 NET "led_out[2]" LOC = P7;
5 NET "led_out[3]" LOC = P6;
6 NET "reset" LOC = P11;
7
8 NET "clk" IOSTANDARD = LVCMOS33;
9 NET "led_out[0]" IOSTANDARD = LVCMOS33;
10 NET "led_out[1]" IOSTANDARD = LVCMOS33;
11 NET "led_out[2]" IOSTANDARD = LVCMOS33;
12 NET "led_out[3]" IOSTANDARD = LVCMOS33;
13 NET "reset" IOSTANDARD = LVCMOS33;
14 NET "clk" SLEW = FAST;
```

◆ LOC: 管脚号引用对应资源
◆ IOSTANDARD: 电平标准
◆ SLEW: 信号翻转速率
fast和Slow(默认)



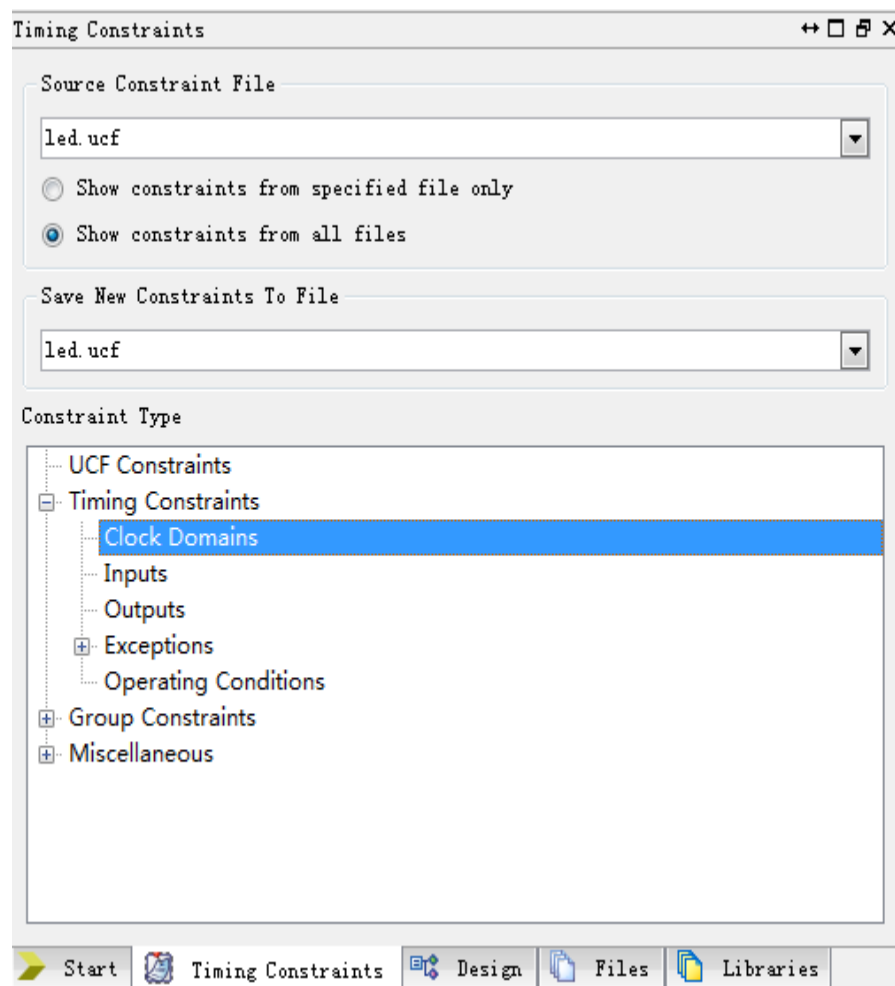
5 创建硬件约束 ——时序约束 (周期约束)

(1) 选中Implementation,点击顶层文件led(led.v), 打开Process窗口, 找到User Constraints, 双击Creat Timinng Constraints,如果出现对话框问是否创建UCF文件,选Yes.



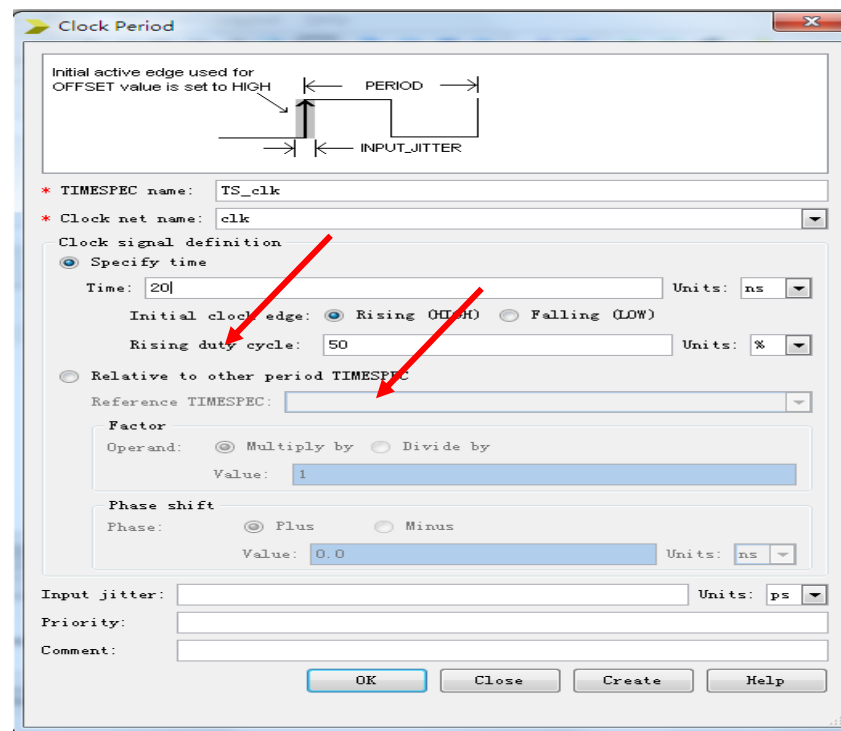
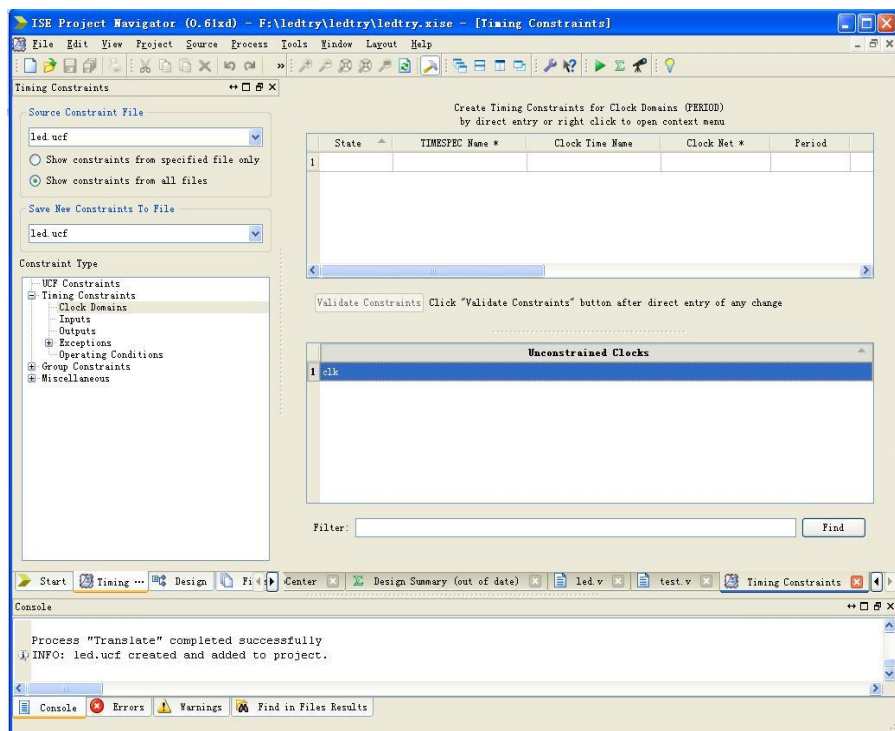
5 创建硬件约束 —— 时序约束 (周期约束)

(2) 在弹出的Constraint Type窗口中选择Clock Domains



5 创建硬件约束 ——时序约束 (周期约束)

(3) 双击clk;打开进入时钟设定界面，设置相应参数，因为开发板提供的系统时钟为50MHz，所以设置时钟的周期为20ns，上升沿有效，占空比设为50%



(4) 点击Create并点击OK，保存后再关闭Timing Constraints界面。 ---完成了系统时钟周期约束

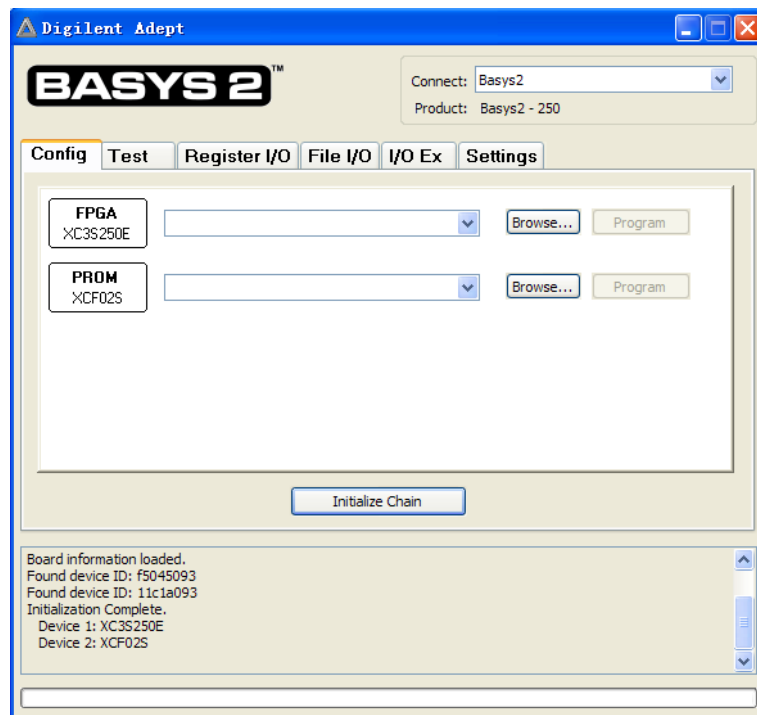
6 综合编译下载

完成约束创建并执行编译并生成下载到硬件需要的.bit文件。

1) USB线连接BASYS2实验板与PC机

打开BASYS2实验板上电源开关

2) Basys2板的专用下载工具Adept下载:开始->**Digilent->Adept ->adept**



四 实验内容与要求

功能要求

组合电路设计、仿真及下载实现按座位号选做

- 1) 一位大小比较器（三灯指示，大于，等于，小于）
- 2) 位2选1数据选择器
- 3) 一位全加器
- 4) 8线-3线优先编码器

设计步骤与要求

- 1) 说明采用Basys2实验板时钟50MHz实现系统功能的基本原理
- 2) 在Xilinx ISE, 编写输入所设计的源程序文件
- 3) 对源程序进行编译及仿真分析
- 4) 管脚约束文件, 编译与逻辑综合, 生成.bit类型文件
- 5) Basys2实验板上下载.bit文件, 验证电路功能

注意事项

◆相关路径及变量不要使用中文！

◆实验报告内容

一、实验任务

二、设计平台

(1) 软件设计流程及作用

(2) 硬件平台简要介绍

三、设计源文件及注释

四、仿真文件及注释

五、仿真波形及分析

六、结果说明

七、思考及总结（选）

详细ISP器件的开发流程-流水灯设计举例

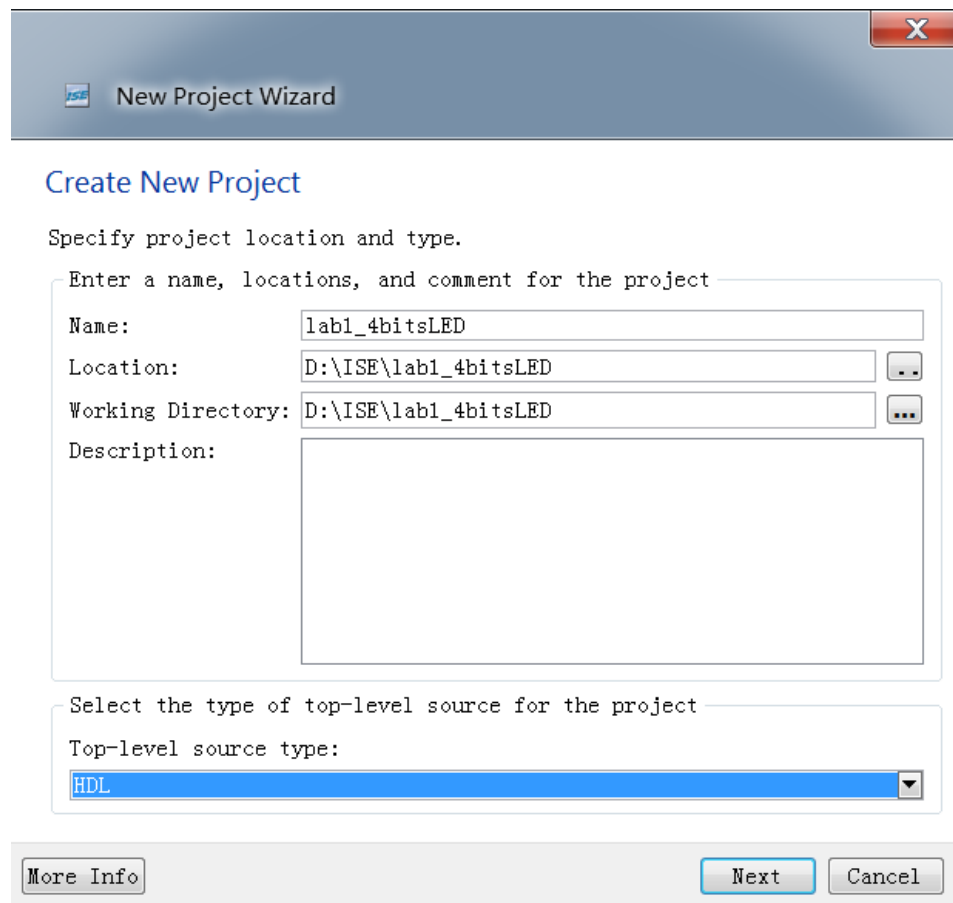
1. 新建工程

(1) 开启ISE14.7软件: 开始→程序→Xilinx ISE Design Suite 14.7→ISE Design Tools→Project Navigator,会出现ISE14.7 的画面.



1. 新建工程

(2) 在 ISE14.7 软件环境下,开启一个新的工程: File → New Project.



New Project Wizard

Create New Project

Specify project location and type.

Enter a name, locations, and comment for the project

Name: lab1_4bitsLED

Location: D:\ISE\lab1_4bitsLED

Working Directory: D:\ISE\lab1_4bitsLED

Description:

Select the type of top-level source for the project

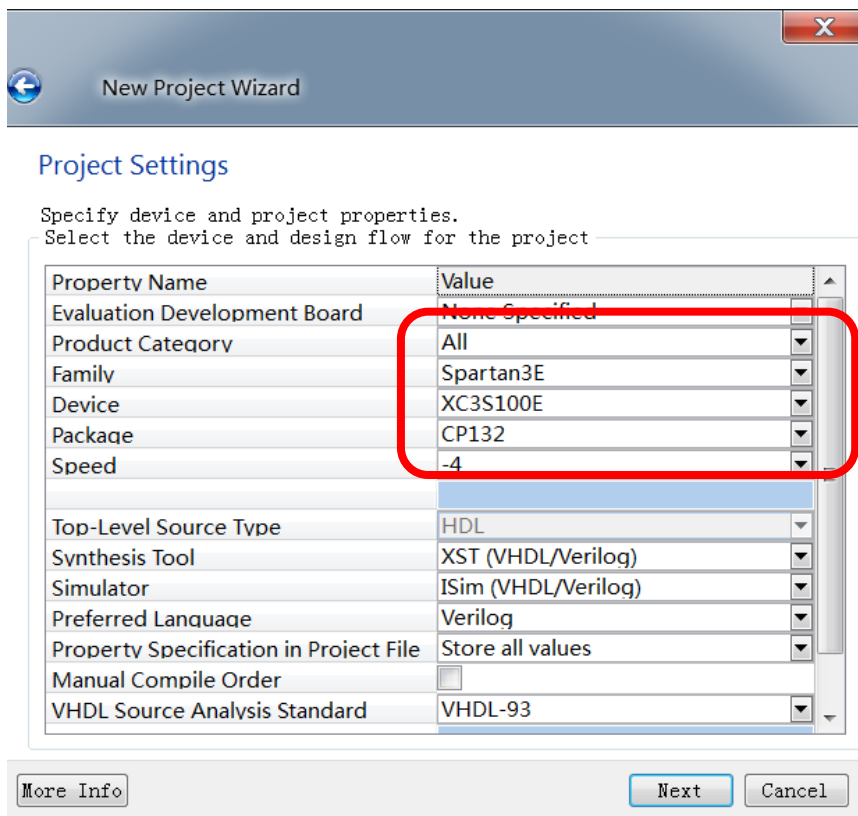
Top-level source type: HDL

More Info Next Cancel

目录及文件名中
不要有中文！也
不要存放到
windows系统
类的目录，比如
桌面、我的文档、
Users等，因为
实际目录可能是
中文的。

1. 新建工程

(3) 单击next,下一个画面就是设定硬件FPGA的参数---请对照实验板芯片系列进行选择



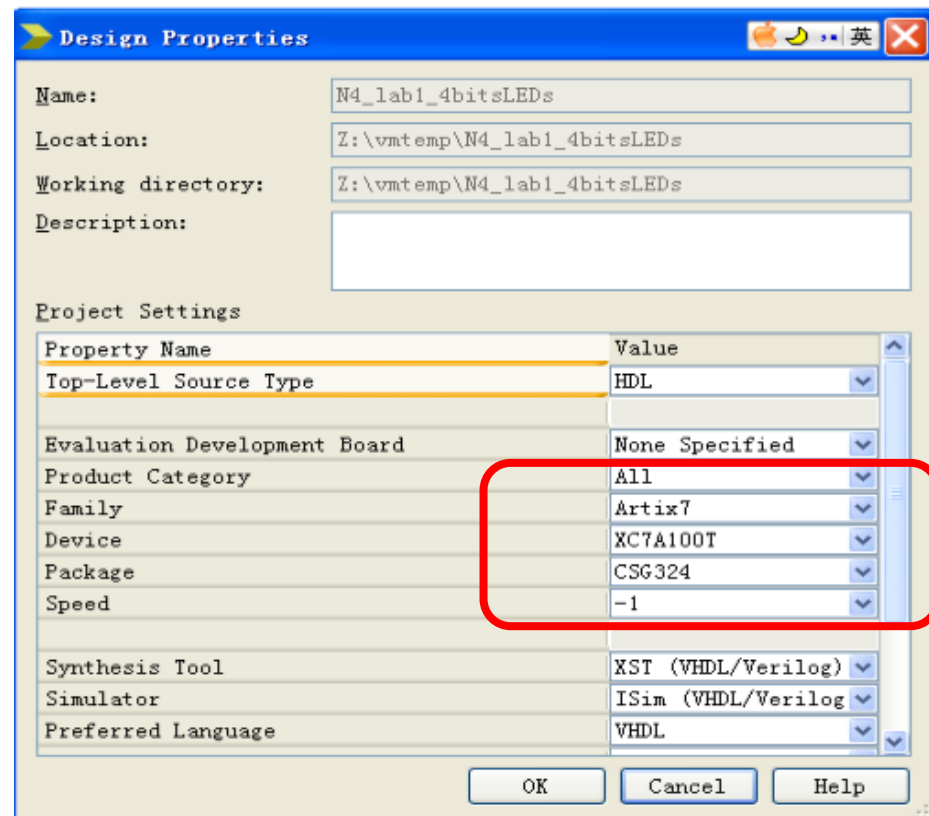
New Project Wizard

Project Settings

Specify device and project properties.
Select the device and design flow for the project

Property Name	Value
Evaluation Development Board	None Specified
Product Category	All
Family	Spartan3E
Device	XC3S100E
Package	CP132
Speed	-4
Top-Level Source Type	HDL
Synthesis Tool	XST (VHDL/Verilog)
Simulator	ISim (VHDL/Verilog)
Preferred Language	Verilog
Property Specification in Project File	Store all values
Manual Compile Order	☐
VHDL Source Analysis Standard	VHDL-93

More Info Next Cancel



Design Properties

Name: N4_lab1_4bitsLEDs

Location: Z:\vmtmp\N4_lab1_4bitsLEDs

Working directory: Z:\vmtmp\N4_lab1_4bitsLEDs

Description:

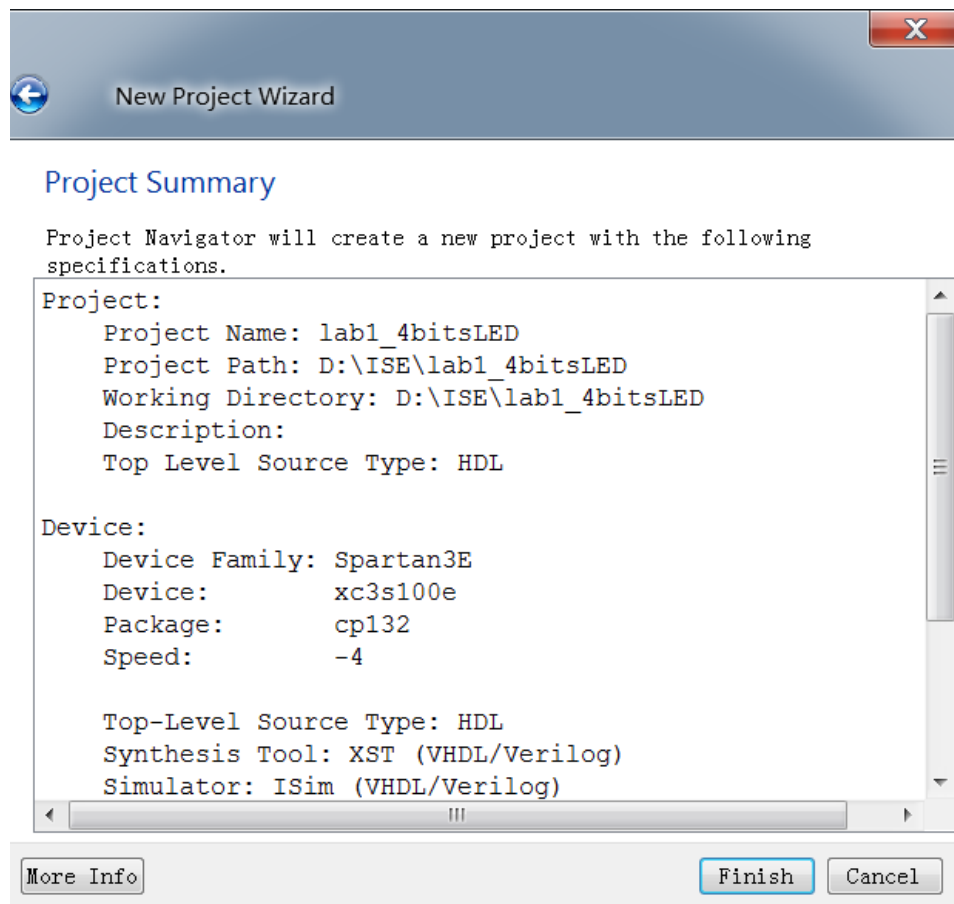
Project Settings

Property Name	Value
Top-Level Source Type	HDL
Evaluation Development Board	None Specified
Product Category	All
Family	Artix7
Device	XC7A100T
Package	CSG324
Speed	-1
Synthesis Tool	XST (VHDL/Verilog)
Simulator	ISim (VHDL/Verilog)
Preferred Language	VHDL

OK Cancel Help

1. 新建工程

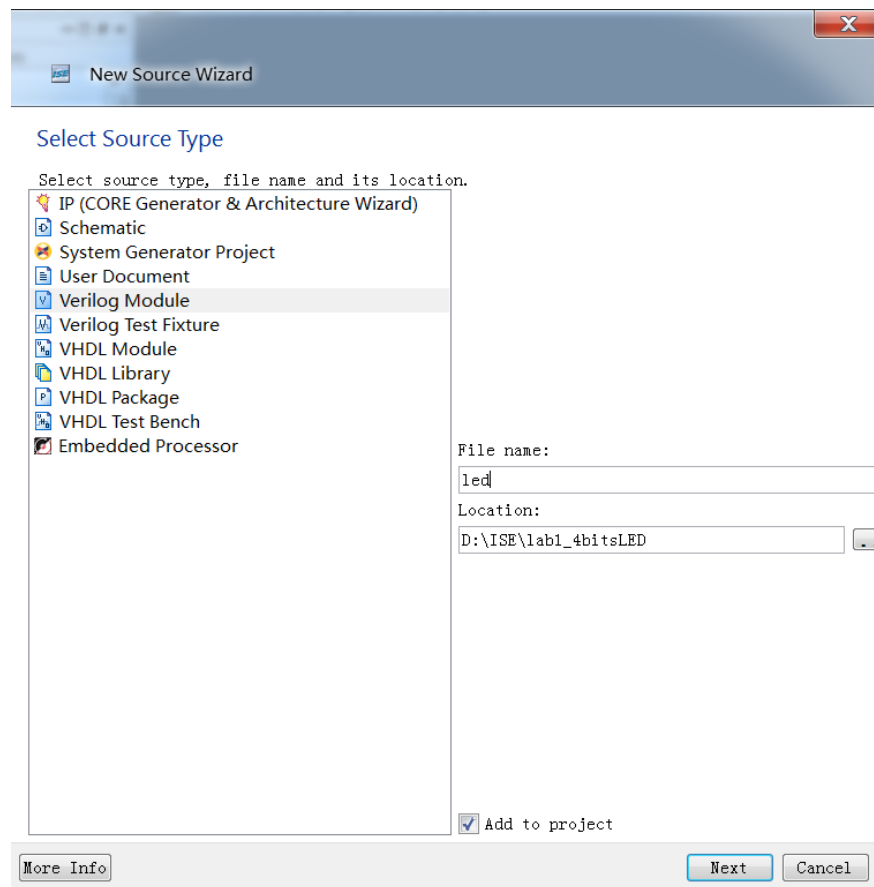
(4) 点击next. 此时出现此项目所有设定的信息, 若需重新设定,则可back. 若无误,则按finish



2.创建新的Verilog源

创建一个新的Verilog源文件

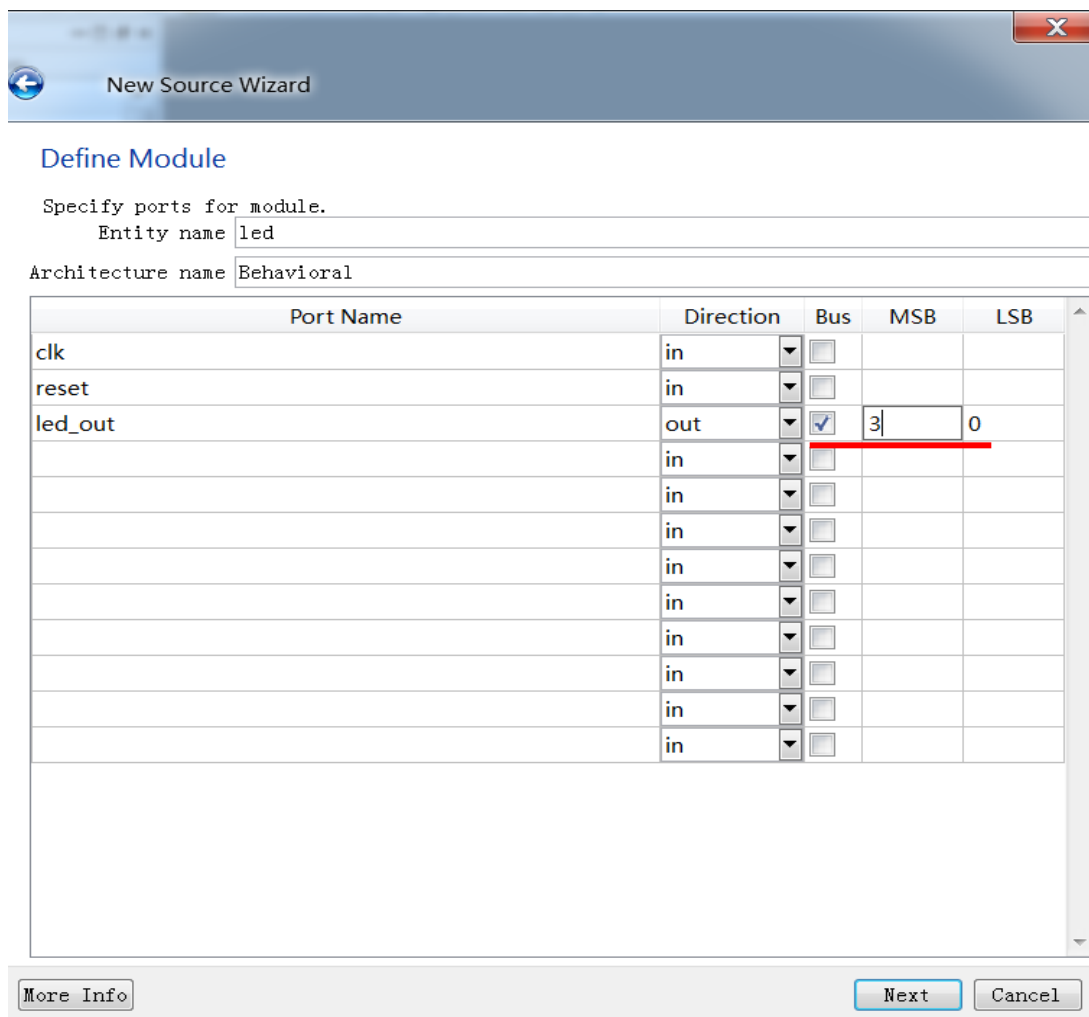
- (1) 此时出现一个项目的框架,可以允许使用者开始进行项目的设计.
- (2) 创建新的设计文件: **Project→New Source**;选择**Verilog Module**,并设定文件名称为**led**



2.创建新的Verilog源

创建一个新的Verilog源文件

(3) 点击**next**,出现**New Source Wizard**,设定此设计的输出输入信号.



The image shows the 'New Source Wizard' dialog box in a software development environment. The title bar says 'New Source Wizard'. Below the title bar, there is a section titled 'Define Module'. Under this section, it says 'Specify ports for module.' There are two input fields: 'Entity name' with the value 'led' and 'Architecture name' with the value 'Behavioral'. Below these fields is a table with columns: 'Port Name', 'Direction', 'Bus', 'MSB', and 'LSB'. The table contains the following rows:

Port Name	Direction	Bus	MSB	LSB
clk	in	☐		
reset	in	☐		
led_out	out	☒	3	0
	in	☐		
	in	☐		
	in	☐		
	in	☐		
	in	☐		
	in	☐		
	in	☐		
	in	☐		
	in	☐		
	in	☐		

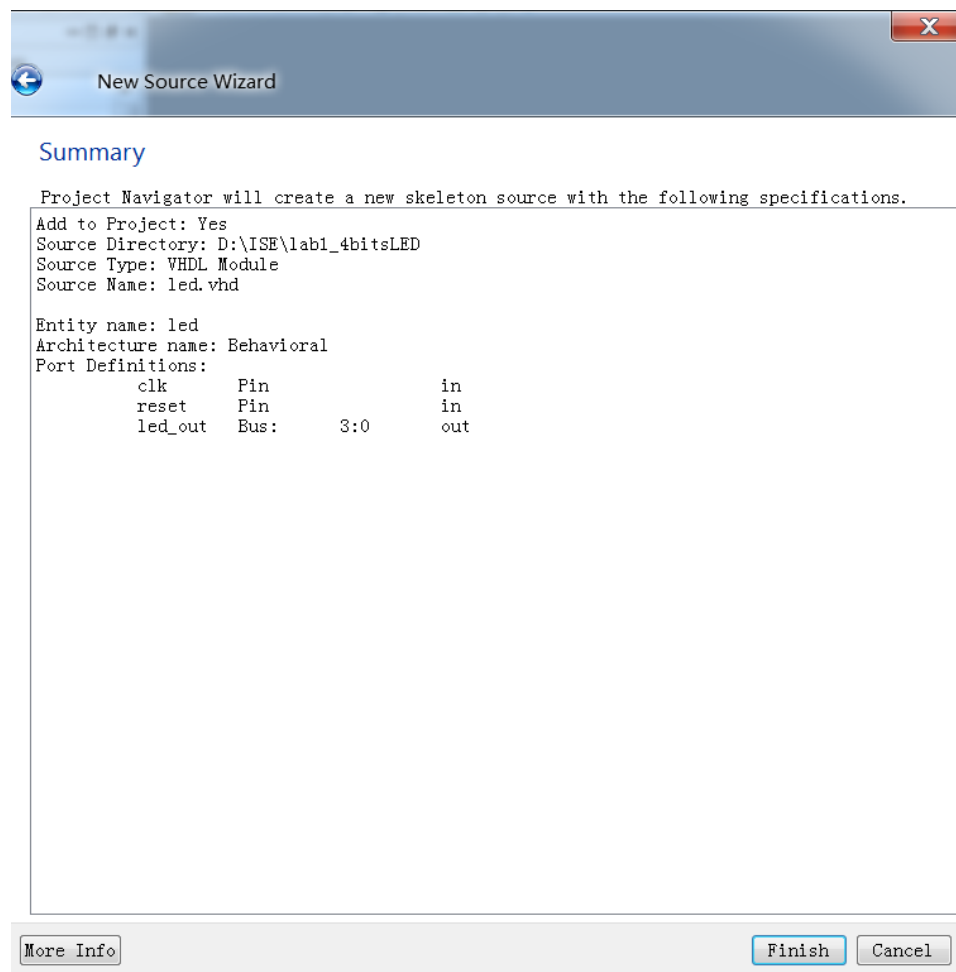
At the bottom of the dialog box, there are three buttons: 'More Info', 'Next', and 'Cancel'.

可以跳过此步，
直接在编写**HDL**
程序中声明。

2.创建新的Verilog源

创建一个新的Verilog源文件

(3) 点击**next**,出现**Summary**,设点击**finish**,完成此设计的输出输入信号.



2.创建新的Verilog源

创建一个新的Verilog源文件

- (4) 按**next**,再按**finish**;此时项目加入此模块之后,在**Sources** 的窗口中会出现**led.v** 的编辑窗口.
- (5) 在Project Navigator右边的工作区可以看到**LED.v**的文件内容,此时可以修改或改变设计内容,在修改完成之后,利用**File→Save**来储存文件.
- (6) 在撰写**LED.v** 内容之时,可以**参考ISE**所附的语言模板Language Template(Edit菜单). 在本实验中我们需要了解计数器模块, 点击软件界面上方的语言模板的快捷键, 然后选择 “Verilog _ Synthesis Constructs _ Coding Examples _ Counters”, 然后选择所需的计数器类型以做参考。

2.创建新的Verilog源文件

实验代码如下：

```
module led(  
    input clk,  
    input reset,  
    output [3:0] led_out );  
reg [26:0] counter;
```

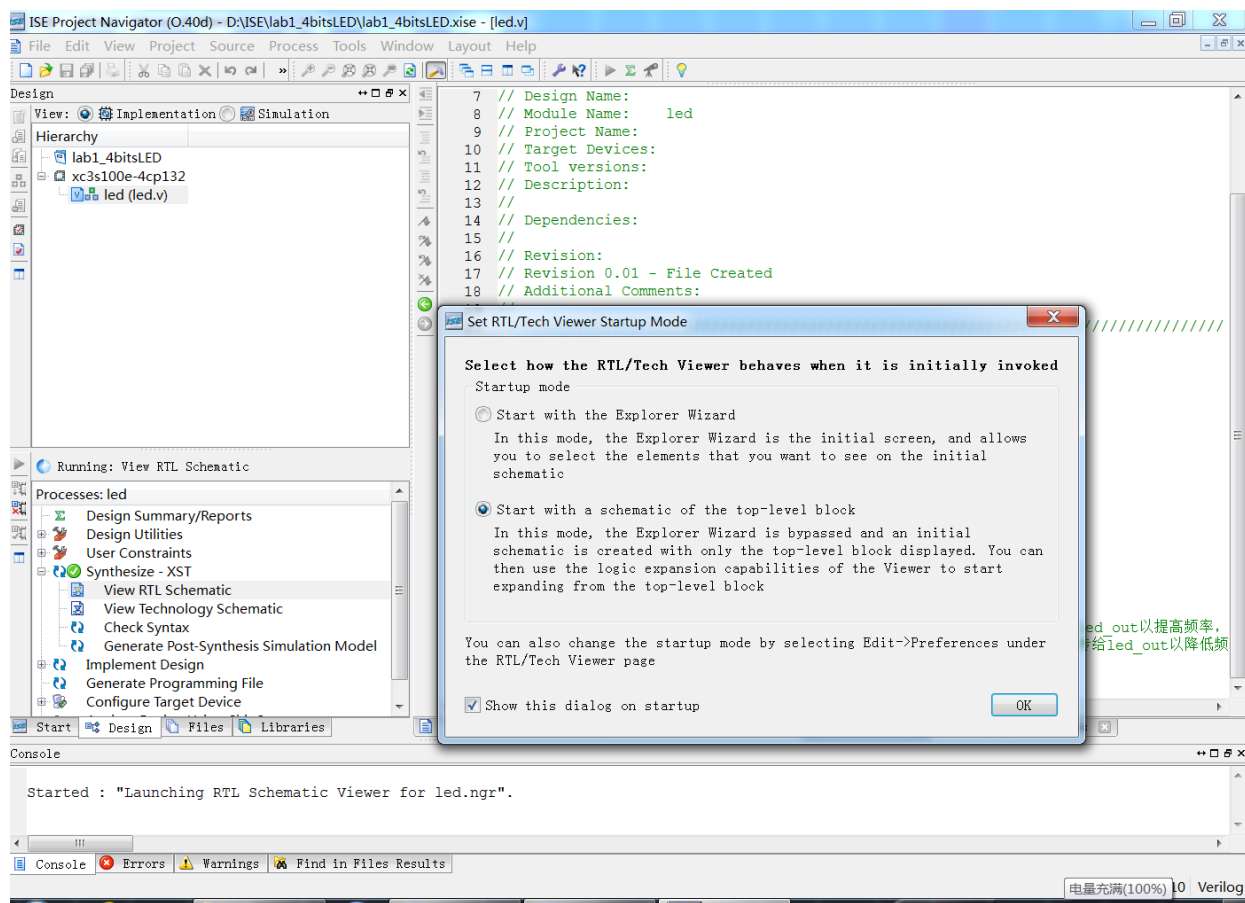
```
    always @(posedge clk)  
        begin  
            if (reset)  
                counter <= 0;  
            else  
                counter <= counter + 1;  
        end
```

```
//assign led_out = counter[3:0];    // 仿真时将counter的低4位传给led_out以提高频率，加快仿真速度  
assign led_out = counter[26:23]; // 实际下载时将counter的高4位传给led_out以降低频率，看到led灯闪烁
```

```
endmodule
```

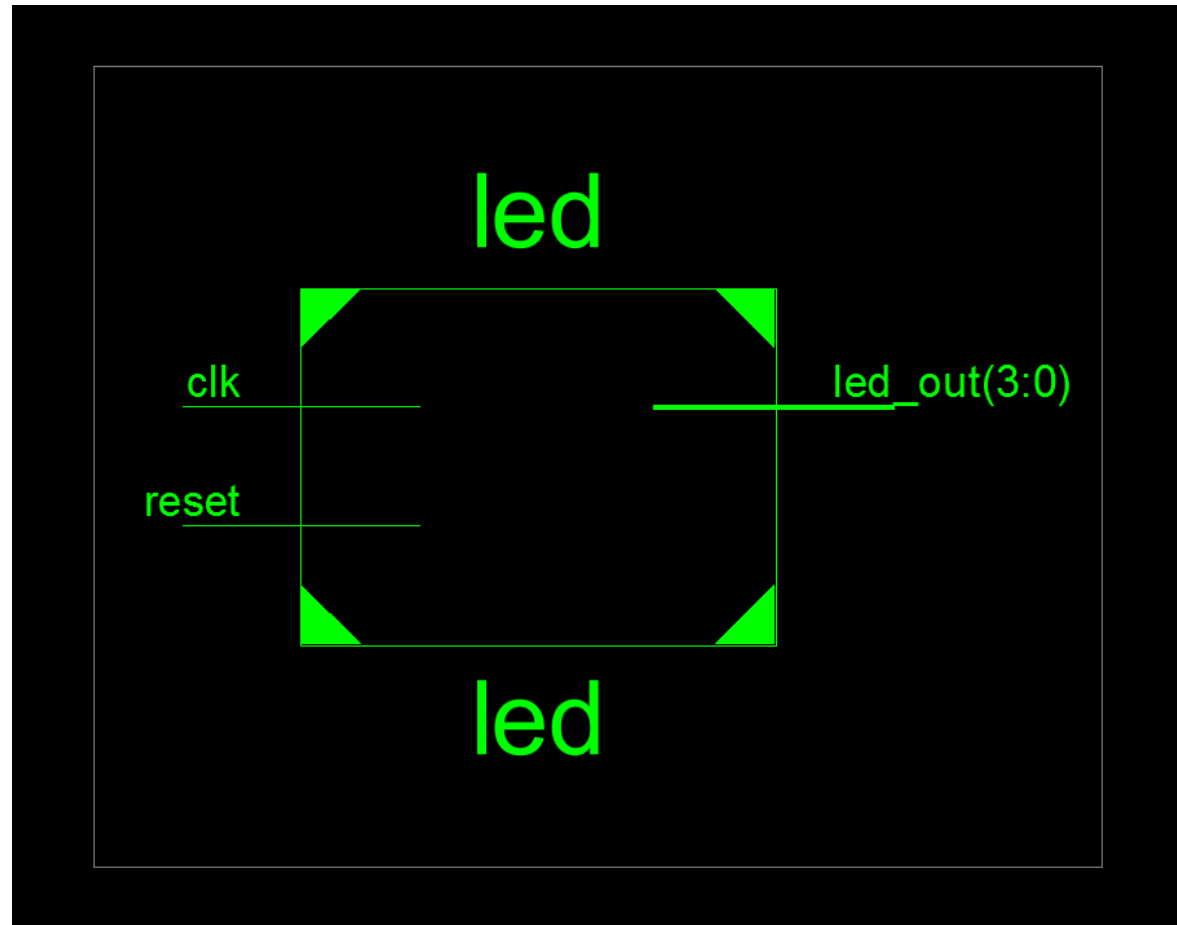
3 编译检错并查看电路

- (1) 双击Synthesize - xst进行编译纠错,以确认设计的正确与否。
- (2) 点开综合选项, 双击View RTL Schematic, 并选择Start with a schematic of the top-level block选项



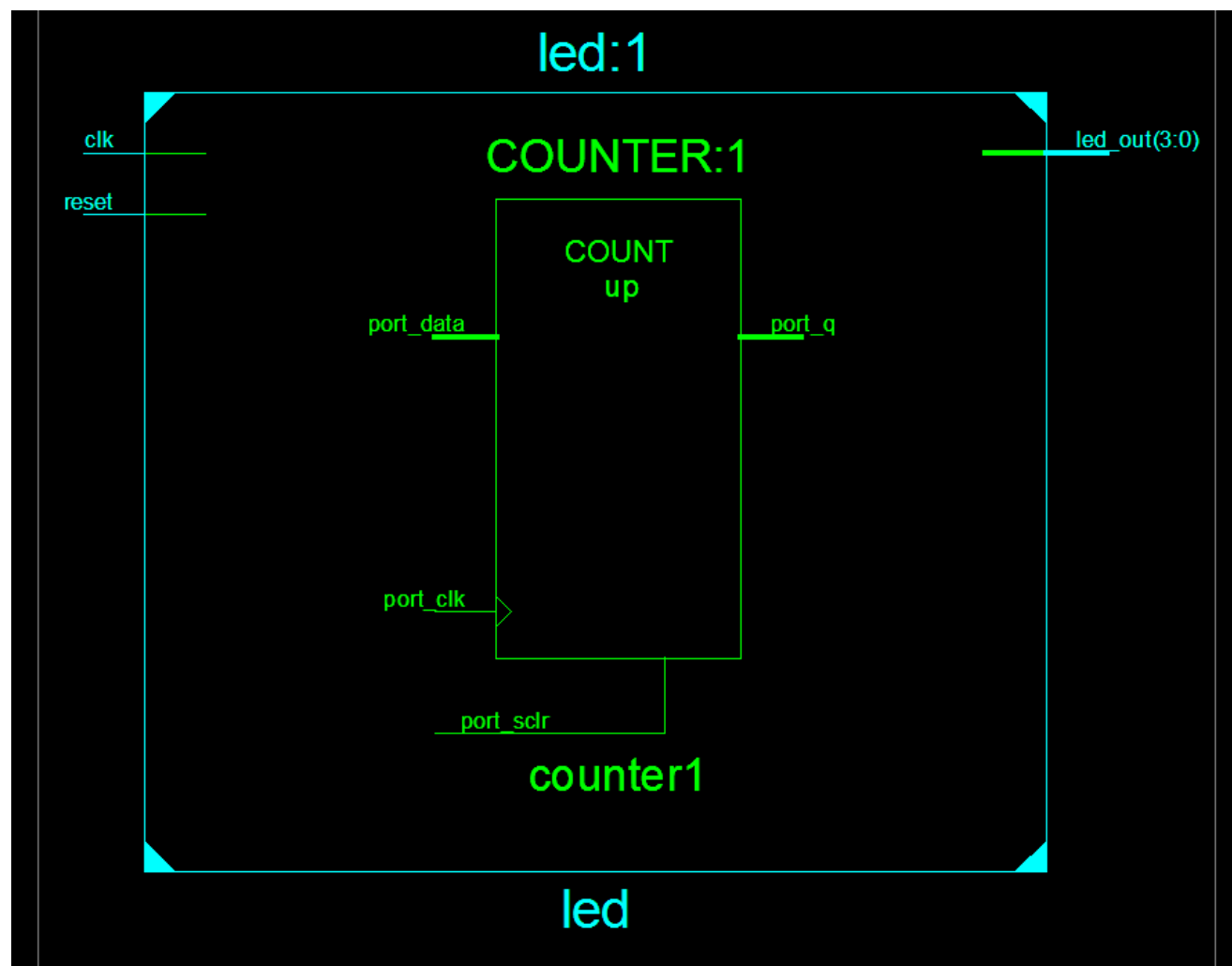
3 编译检错并查看电路

(3) 点击OK, 出现设计的整个RTL(Register Transfer Level,寄存器传输级)电路模块图



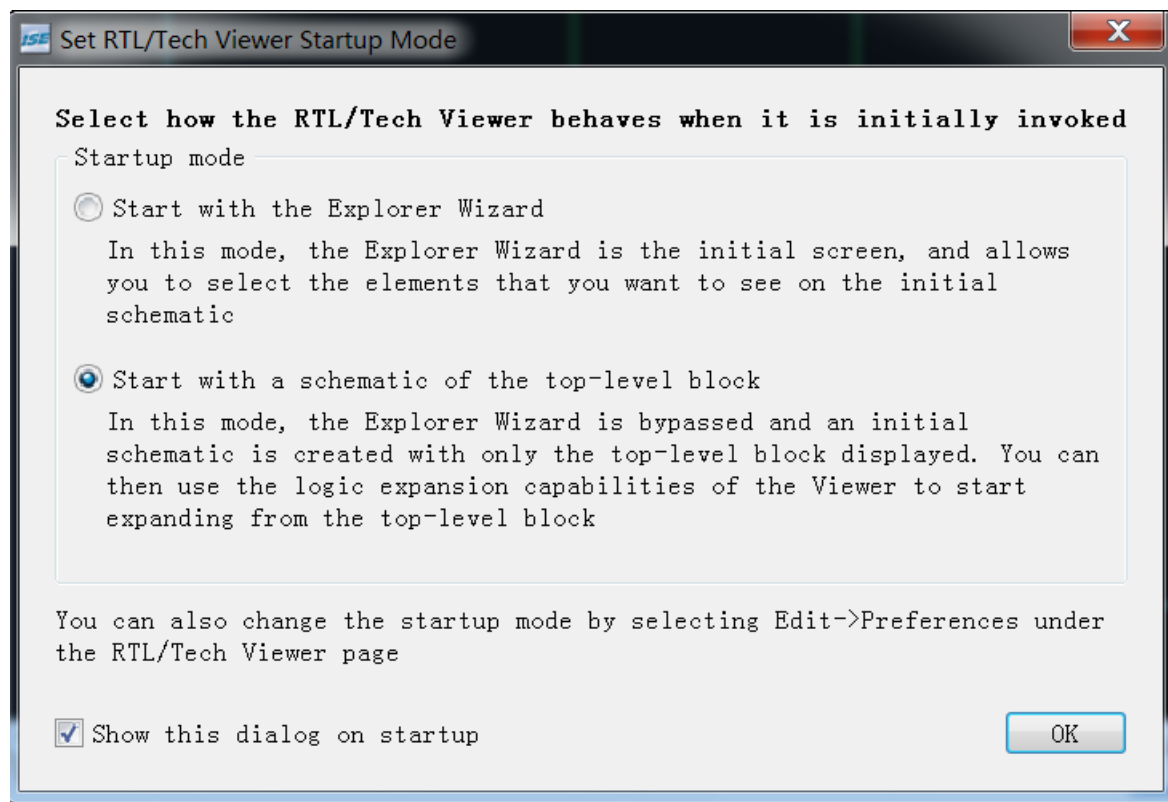
3 编译检错并查看电路

(4) 直接双击电路顶层，查看内部RTL寄存器电路模块



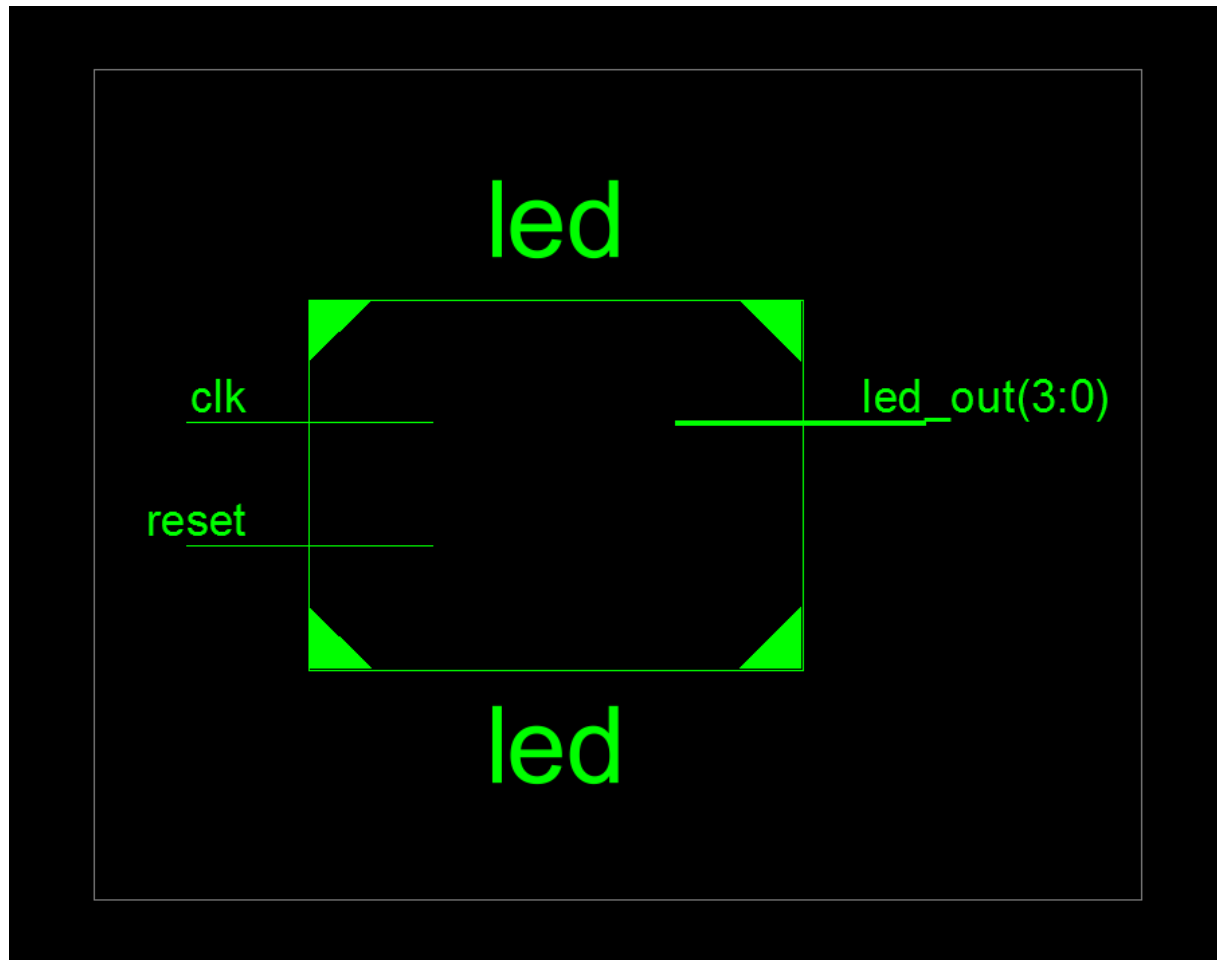
3 编译检错并查看电路

(5) 如果需要查看设计内部具体有哪些实际资源组成，则可以点开综合选项，双击“view technology schematic”



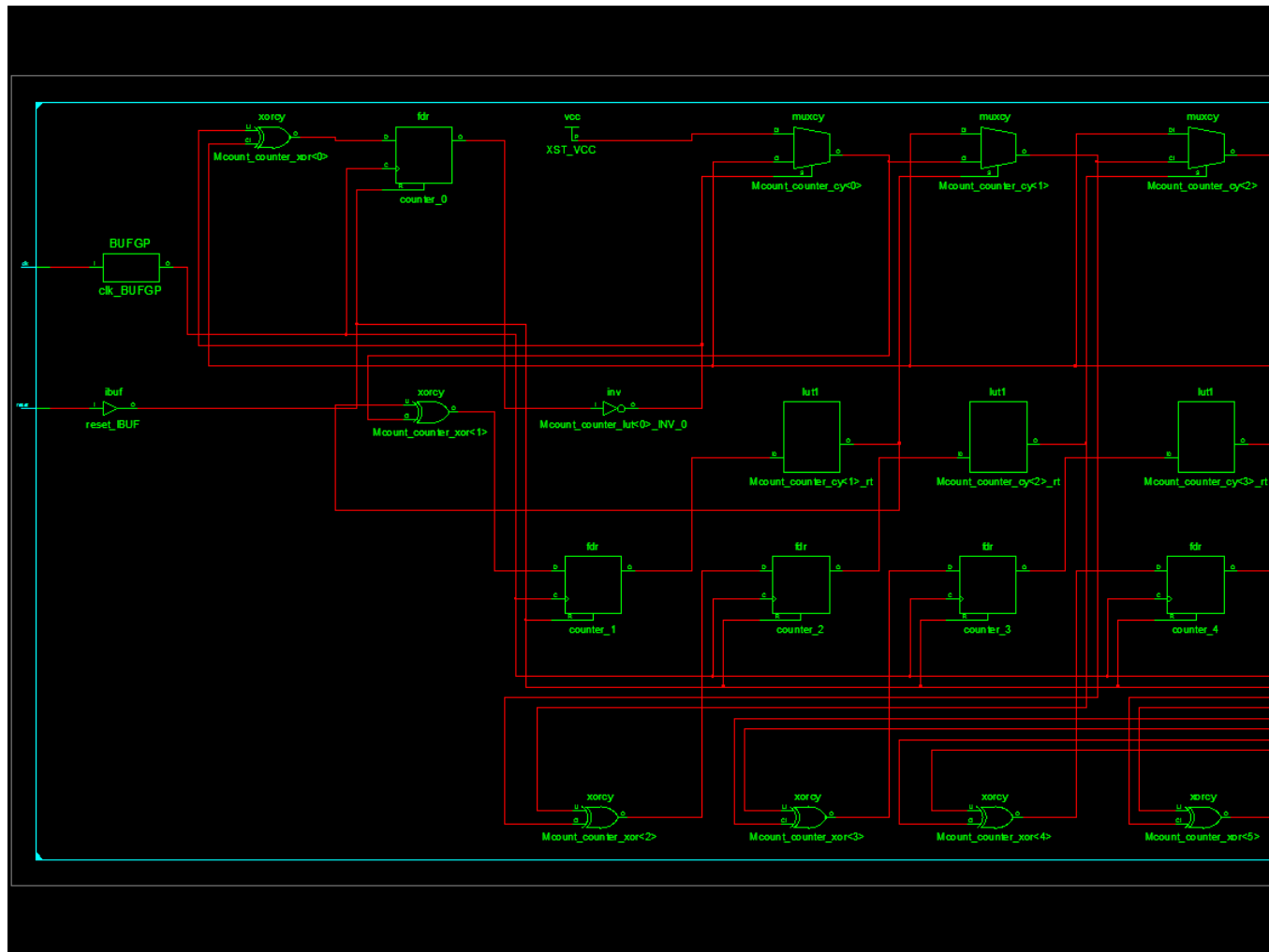
3 编译检错并查看电路

(6) 点击OK，出现设计顶层门级电路图



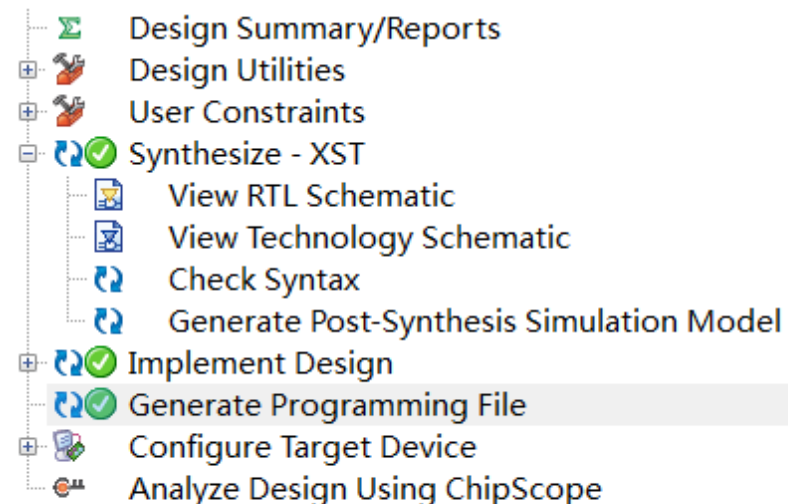
3 编译检错并查看电路

(7) 直接双击顶层电路，则可以看到设计的**内部门级电路**是由哪些资源组成的。



3 编译检错并查看电路

(8) 在Processes的窗口中,直接以鼠标双击 **Generate Programming File** 的选项.此时ISE 会自动执行并产生可以下载的.**.bit** 类型文件,此步骤是最直接验证设计工作的正确性与否.---可以等待仿真验证后再执行。



(9) 若在每一个步骤后都出现绿色的打勾,代表程序成功跑完而没有错误和警告。若有黄色的警告,一般可以忽略。若有一个程序都出现红色打叉[X]的符号,代表有错误,可以依显示结果来侦错. ---可以等待仿真验证后再执行。

4 设计仿真

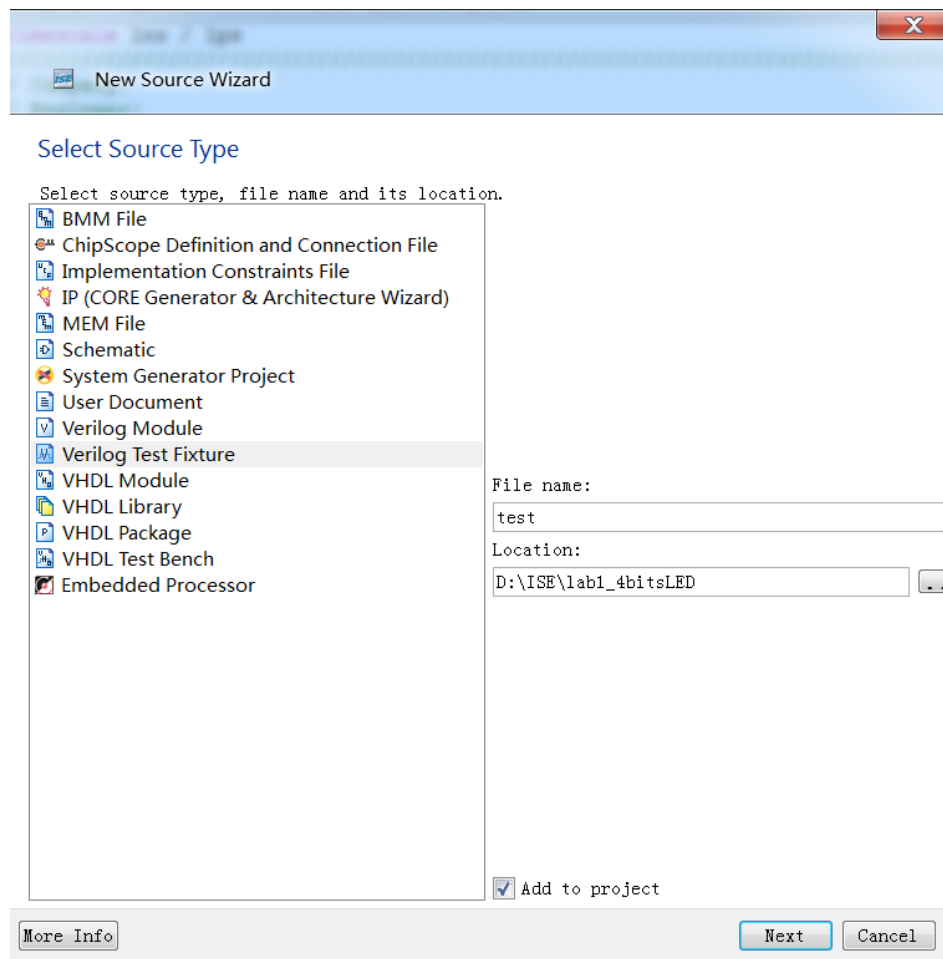
在实际烧录FPGA之前，为了验证设计的正确性,可以先利用测试模板(Testbench)来验证设计的正确性。在这里请注意一下，为了加快仿真进程，仿真

```
37 assign led_out = counter[3:0]; // 仿真时将counter的低4位传给led_out以提高频率，加快仿真速度
38 // assign led_out = counter[26:23]; // 实际下载时将counter的高4位传给led_out以降低频率，看到led灯闪烁
```

快地看到输出仿真结果。这时，将代码led.v的第37行使能，第38行不使能，然后保存。

4 设计仿真

(1) 点击**led** HDL文件,创建一个新的测试平台源文件:
Project → New Source.在源文件向导里,点击**Verilog Test Fixture** 作为源文件类型,输入文件名称为**test**.



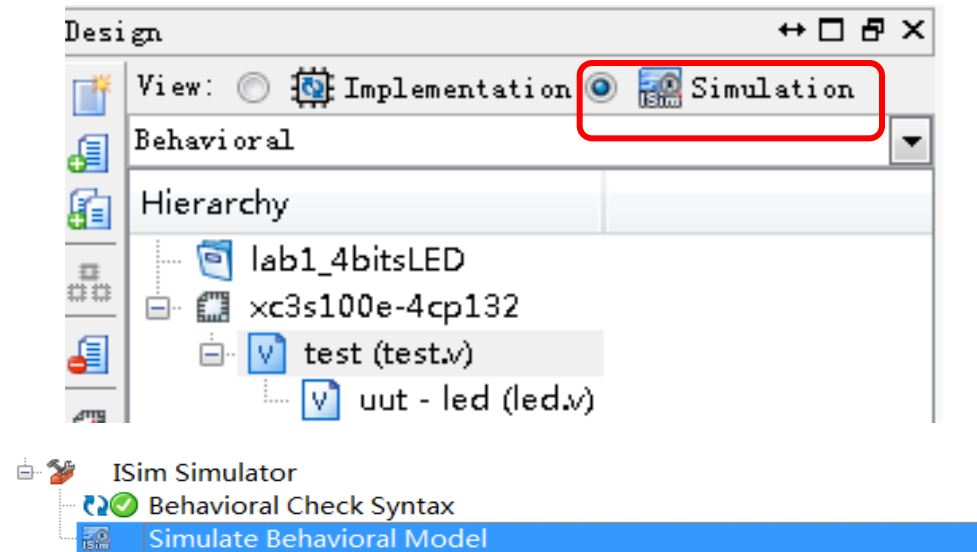
4 设计仿真

(2) 一直点击**next**,直到点击**finish**,自动生成**test.v**的测试模板,在此基础上编辑输入激励: 时钟周期设定为10ns, 复位信号为高持续500ns后, 再将复位信号置低。

```
24
25 module test;
26
27     // Inputs
28     reg clk;
29     reg reset;
30
31     // Outputs
32     wire [3:0] led_out;
33
34     parameter PERIOD = 10; //设定时钟周期为10ns
35
36     // Instantiate the Unit Under Test (UUT)
37     led uut (
38         .clk(clk),
39         .reset(reset),
40         .led_out(led_out)
41     );
42
43     //生成时钟
44     always begin
45         clk = 1'b0;
46         #(PERIOD/2) clk= 1'b1;
47         #(PERIOD/2);
48     end
49
50     initial begin
51         // Initialize Inputs
52         clk = 1'b0;
53         reset = 1;
54
55         // Wait 500 ns for global reset to finish
56         #500;|
57
58         // Add stimulus here
59         reset = 0;
60
61     end
62
63 endmodule
64
65
```

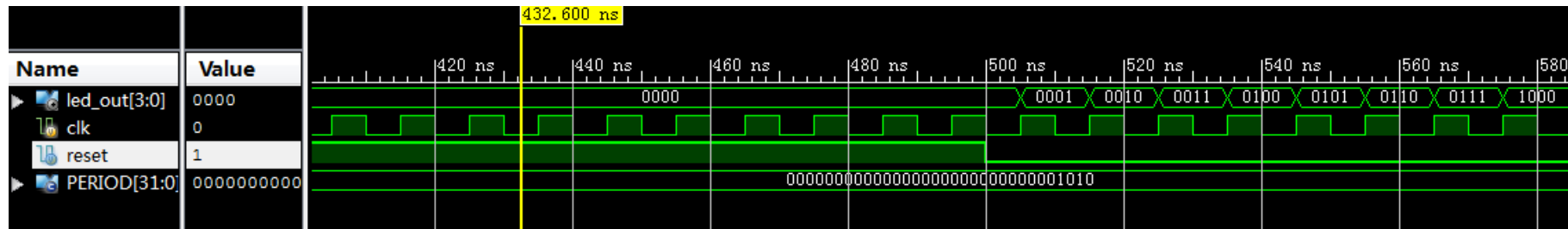
4 设计仿真

(3) 保存test.v。选择sources for Simulation
双击“Behavior Check syntax”。



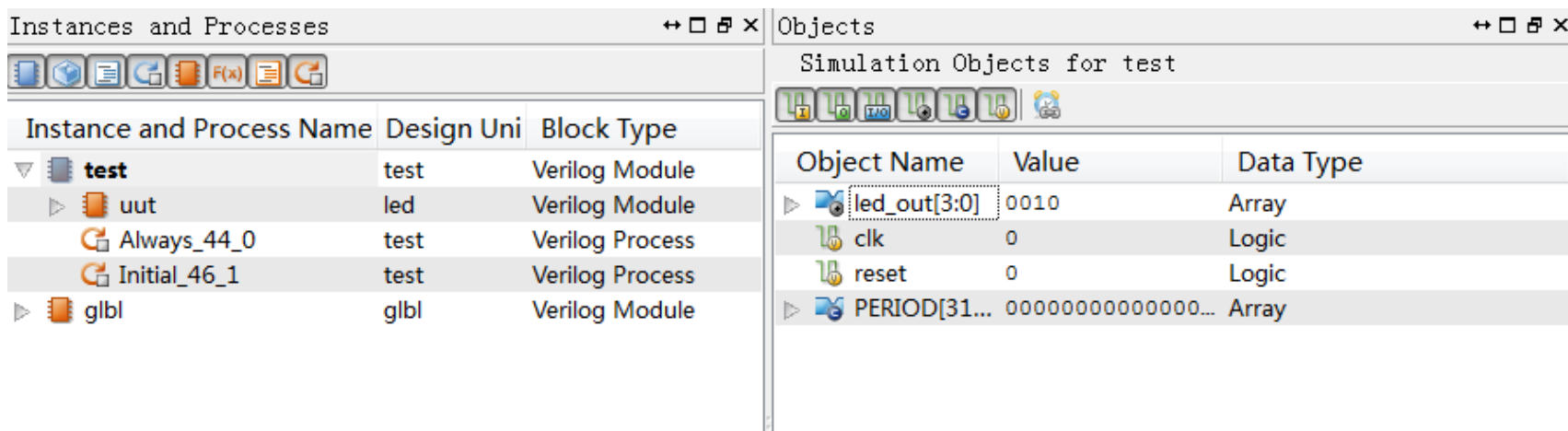
4 设计仿真

(4) 双击Simulate Behavioral Model ,ISE仿真器打开并开始仿真, ,将图形界面缩小到合适的界面,其仿真结果如图所示:



4 设计仿真

(5) 我们也可以查看设计的内部信号。添加内部信号的步骤是：在Isim的Instances and Processes窗口中点选**test**,然后点击UUT，在object窗口则会出现全部信号：

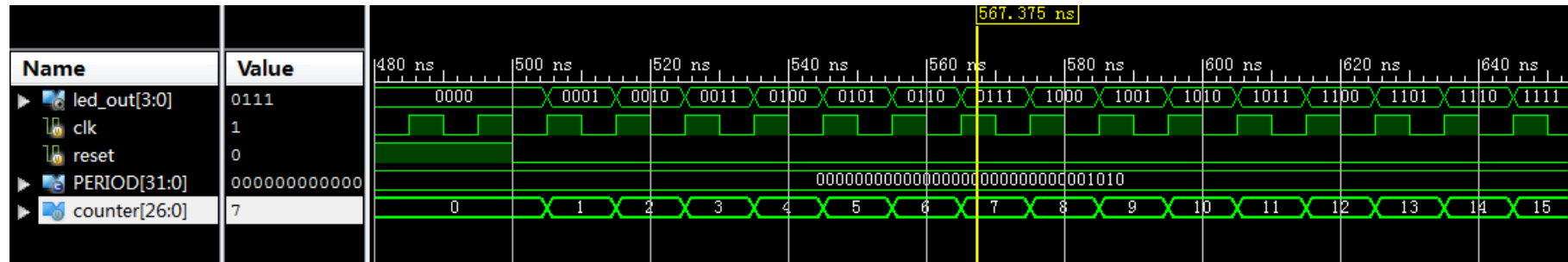


The screenshot displays two windows from the Xilinx ISE simulation environment. The 'Instances and Processes' window on the left shows a hierarchical tree of the design. The 'test' instance is expanded, revealing its components: 'uut' (Verilog Module), 'Always_44_0' (Verilog Process), 'Initial_46_1' (Verilog Process), and 'glbl' (Verilog Module). The 'Objects' window on the right shows the 'Simulation Objects for test'. It contains a table of internal signals and their current values.

Object Name	Value	Data Type
led_out[3:0]	0010	Array
clk	0	Logic
reset	0	Logic
PERIOD[31...	0000000000000000...	Array

4 设计仿真

(6) 拖动counter到仿真波形里, 点击restart按钮, 再点击run all按钮运行一段时间后点击暂停, 就可以看到内部信号仿真图。为了查看方便, 将二进制数改为10进制无符号数的形式, 选择counter信号, 点击右键选择radix扩展栏中的Unsigned Decimal, 由图可知counter信号是在正确计数的。

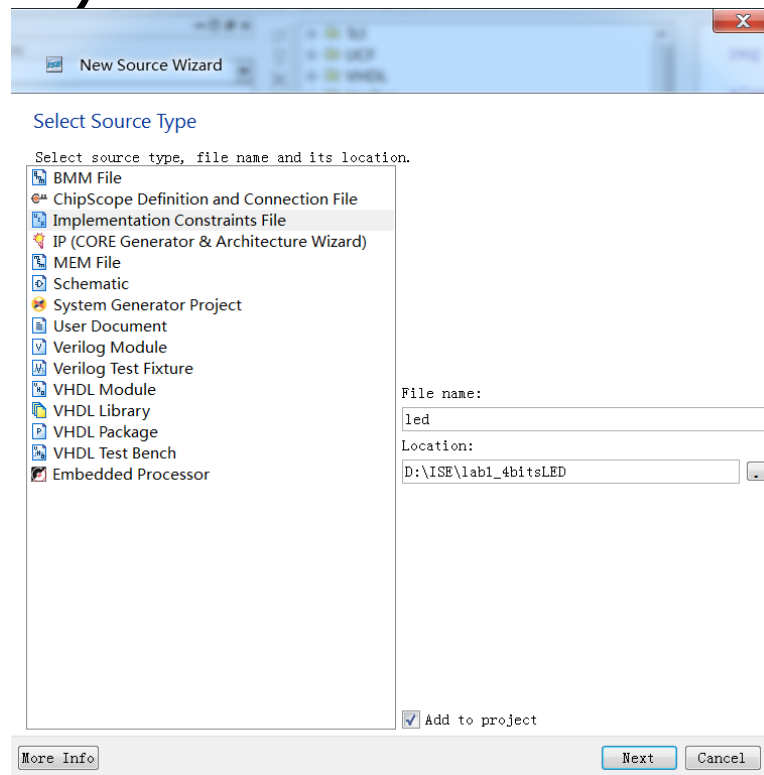


(7) 关闭Isim仿真器并保存。

5 创建约束

(1) 设定I/O 脚的位置,可以利用LED.UCF 来设定I/O 脚的位置,以得到正确的输出文件.

Project → New Source. 设定输入的文件格式为 **Implementation Constraints File**, 文件名称为led(自动储存为led.UCF)

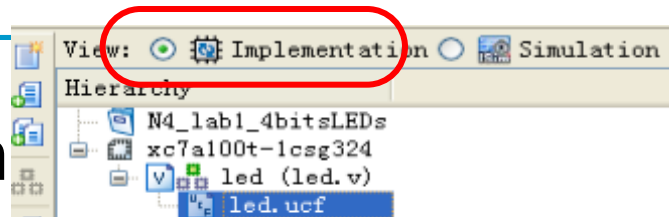


5 创建约束

(2) 一直点击**next**，最后按**finish**

(3) 在**Design**选择**Implementation**

(4) 点选**Sources**窗口内的**led.ucf**, 点击



processess窗口里的**user constraints**，双击**edit constraints (txt)**，输入后储存文件。其中，“**LOC**”代表管脚定义。

“**IOSTANDARD**”代表电平标准，
实验中设定为**LVC MOS33**；

“**SLEW**”代表信号的翻转速率，有
fast和**slow**之分，默认是**slow**，时
钟信号**clk**变化比较快可设定为**fast**。

```
## Clock signal
NET "clk" LOC = "E3" | IOSTANDARD = "LVCMOS33";
#NET "clk" TNM_NET = sys_clk_pin;
#TIMESPEC TS_sys_clk_pin = PERIOD sys_clk_pin 100 MHz HIGH 50%;
## Switches
NET "reset" LOC = "U9" | IOSTANDARD = "LVCMOS33"; #Ba
#NET "sw<1>" LOC = "U8" | IOSTANDARD = "LVCMOS33";
#NET "sw<2>" LOC = "R7" | IOSTANDARD = "LVCMOS33";
#NET "sw<3>" LOC = "R6" | IOSTANDARD = "LVCMOS33";
#NET "sw<4>" LOC = "R5" | IOSTANDARD = "LVCMOS33";
#NET "sw<5>" LOC = "V7" | IOSTANDARD = "LVCMOS33";
#NET "sw<6>" LOC = "V6" | IOSTANDARD = "LVCMOS33";
#NET "sw<7>" LOC = "V5" | IOSTANDARD = "LVCMOS33";
#NET "sw<8>" LOC = "U4" | IOSTANDARD = "LVCMOS33";
#NET "sw<9>" LOC = "V2" | IOSTANDARD = "LVCMOS33";
#NET "sw<10>" LOC = "U2" | IOSTANDARD = "LVCMOS33";
#NET "sw<11>" LOC = "T3" | IOSTANDARD = "LVCMOS33";
#NET "sw<12>" LOC = "T1" | IOSTANDARD = "LVCMOS33";
#NET "sw<13>" LOC = "R3" | IOSTANDARD = "LVCMOS33";
#NET "sw<14>" LOC = "P3" | IOSTANDARD = "LVCMOS33";
#NET "sw<15>" LOC = "P4" | IOSTANDARD = "LVCMOS33";

## LEDs
NET "led_out<0>" LOC = "T8" | IOSTANDARD = "LVCMOS33";
NET "led_out<1>" LOC = "V9" | IOSTANDARD = "LVCMOS33";
NET "led_out<2>" LOC = "R8" | IOSTANDARD = "LVCMOS33";
NET "led_out<3>" LOC = "T6" | IOSTANDARD = "LVCMOS33";
```

5 创建约束

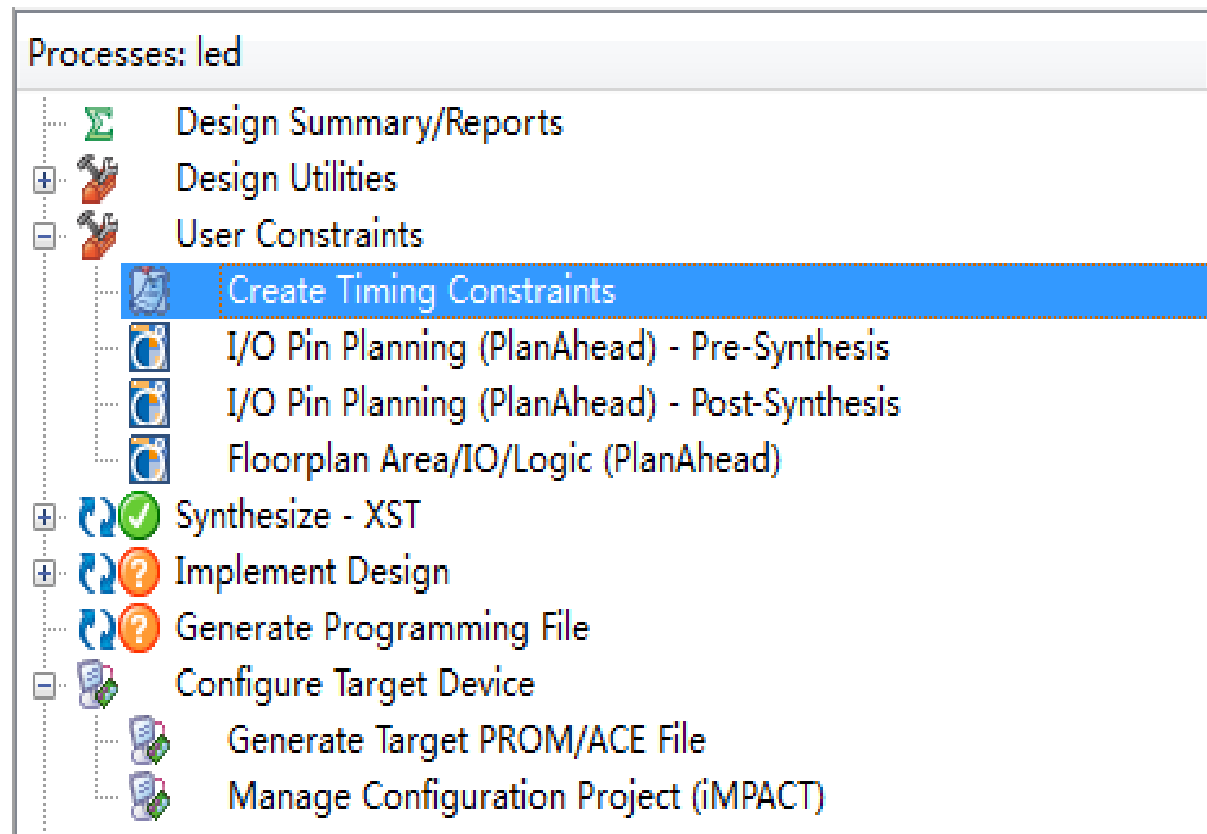
(4) 在执行设计之前，需将之前我们仿真时修改的代码给还原，将代码**led.v**的第**37**行去能，第**38**行使能，然后保存。

(5) 重新执行**Generate Programming File**, 确认设计无误（即确认所有步骤后均出现绿色的打勾,表示无错或至少没有步骤出现红色打叉**[X]**的符号,表示没有不可容忍的错误）。

5 创建约束 ——时序电路需要**时序约束**

选中Implementation，点击顶层文件.v， Process--User Constraints--Creat **Timing Constraints**，如果出现对话框问是否创建约束.ucf文件， Yes.

界面基于Xilinx
ISE开发软件举例，
其他FPGA开发软件界面不同，但
概念类似。

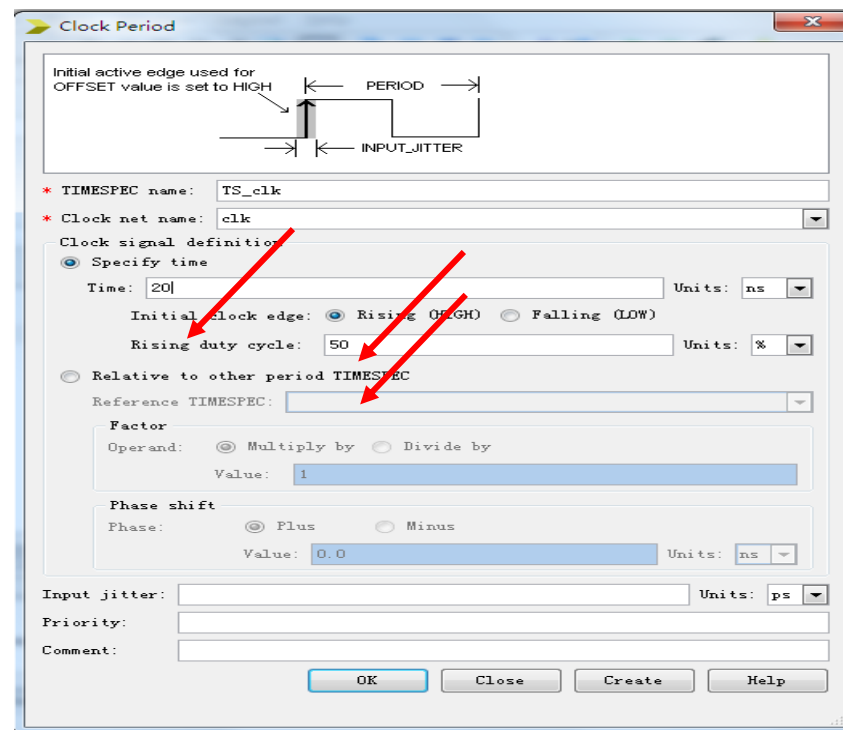
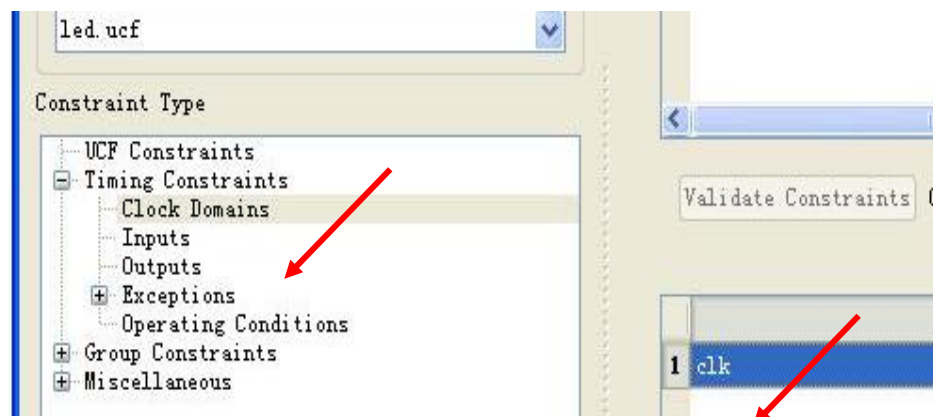


5 创建约束 —— 时序电路需要时序约束

在Constraint Type弹出窗中选择 **Clock Domains** 双击主设计的时钟端口名（比如CLK），打开进入时钟设定界面。

因为N4开发板提供的系统时钟为**100MHz**，所以设置时钟周期Time为**10ns**，再点选

上升沿Rising (High)
占空比duty Cycle设 50%



5 创建约束 ——时序电路需要**时序约束**

点击Create并点击OK，保存后再关闭Timing Constraints界面。---即**完成系统时钟周期约束**

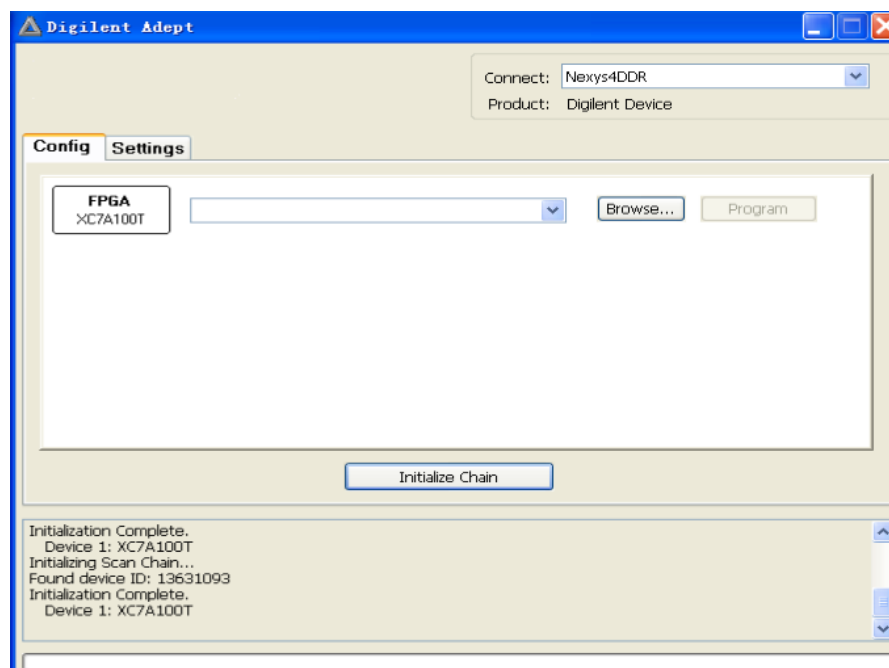
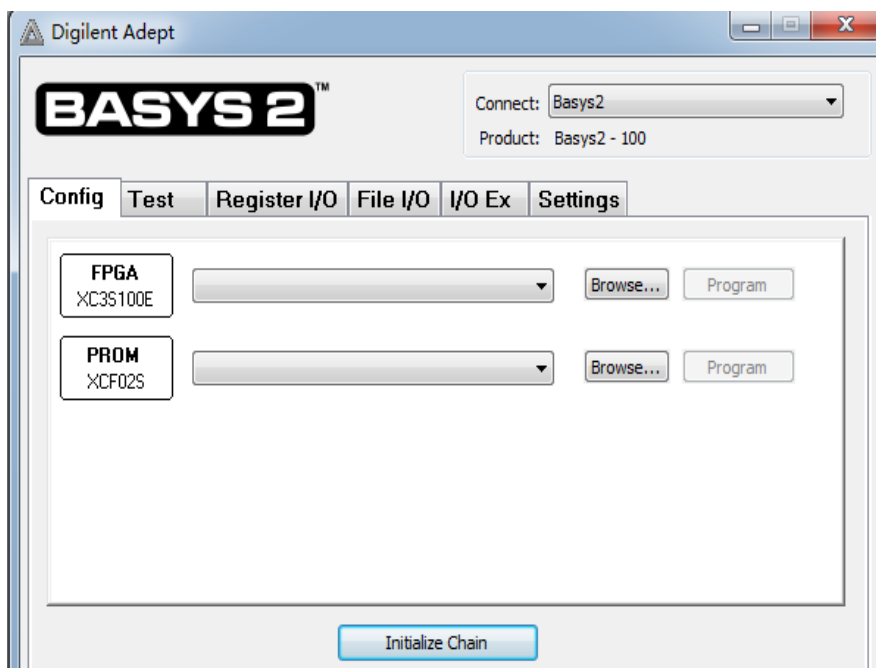
上述操作会存到 .ucf中，内容类似下面文本的**前几行**，配合后面管脚约束操作的**时钟管脚约束（末行）**，保证系统时钟正确加入所设计工程。

```
## Clock signal
NET "clk" TNM_NET = sys_clk_pin;
TIMESPEC TS_sys_clk_pin = PERIOD sys_clk_pin 100 MHz
HIGH 50%;

NET "clk" LOC = "E3" | IOSTANDARD = "LVCMOS33";
```

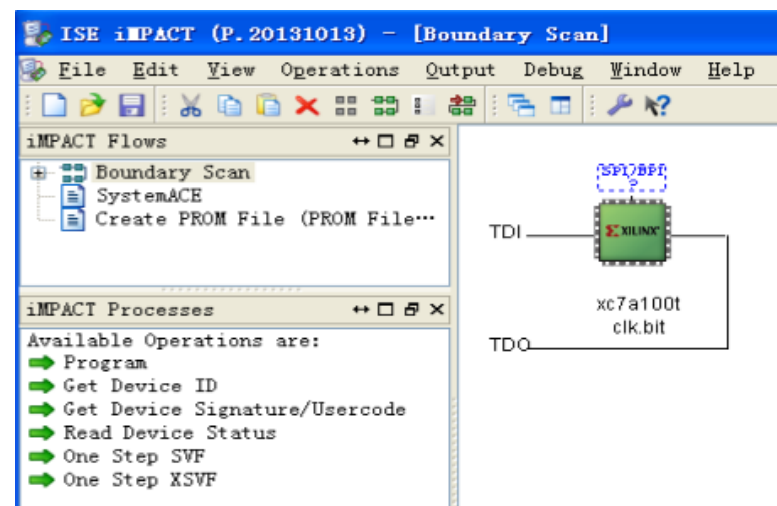
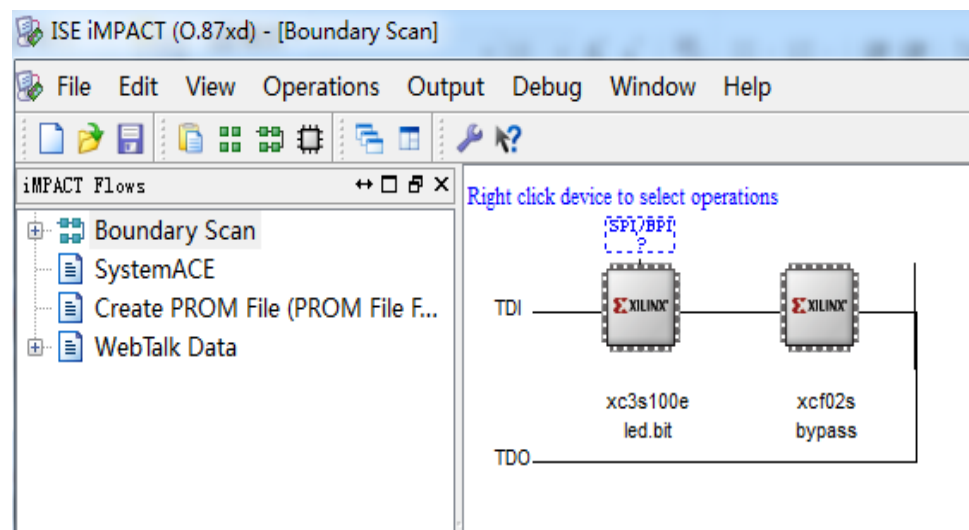
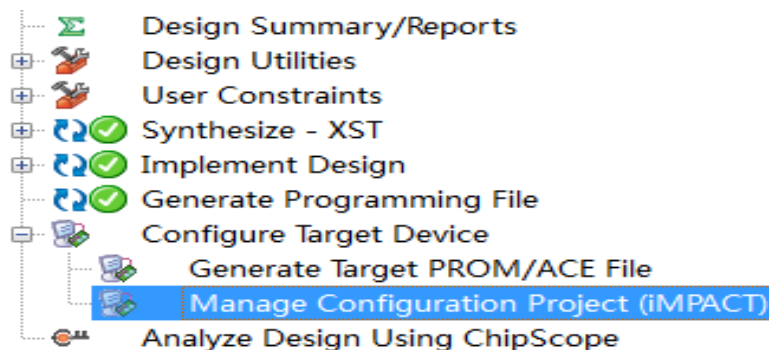
6 下载设计到实验板

(1) 在完成设计验证之后,就可以将**led.bit** 写入到**FPGA** 内部,此时连接**BASYS2**实验板的**USB**下载线到**PC**机, 并打开**BASYS2**实验板上的电源开关, 在**PC**机桌面点击“开始->**Digilent->Adept ->adept**”, 打开专门的下载工具。



6 下载设计到实验板

(2) 也可以用**ISE**自带的工具下载，详见**DOC**文档。



1Hz产生电路的设计

50MHz

1Hz产生电路

1Hz

```
//***** clk_1Hz.v *****
```

```
module clk_1Hz (clk_50M, clk_1hz);
```

```
    input   clk_50M;
```

```
    output clk_1hz;
```

```
    reg [24:0] Cnt;           //计50MHz的个数25M=17D7840
```

```
    reg clk_1hz;
```

```
always @(posedge clk_50M)
```

```
    begin
```

```
        if(Cnt>25'd25000000)
```

```
            begin
```

```
                Cnt=0;
```

```
                clk_1hz=~clk_1hz;
```

```
            end
```

```
            else Cnt=Cnt+1;
```

```
    end
```

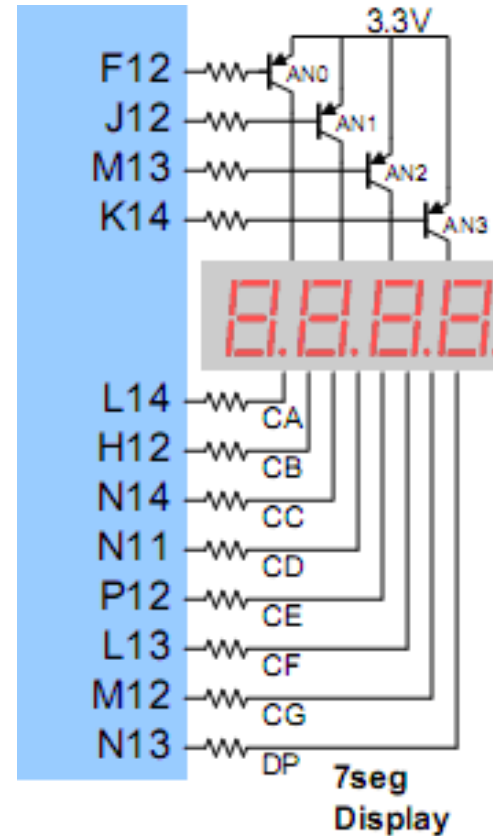
```
endmodule
```

```

■ always @ (disp_dat)
■ begin
■     case(disp_dat)
■         4'h0: segdat_reg = 7'b0000001; //0
■         4'h1: segdat_reg = 7'b1001111; //1
■         4'h2: segdat_reg = 7'b0010010; //2
■         4'h3: segdat_reg = 7'b0000110; //3
■         4'h4: segdat_reg = 7'b1001100; //4
■         4'h5: segdat_reg = 7'b0100100; //5
■         4'h6: segdat_reg = 7'b0100000; //6
■         4'h7: segdat_reg = 7'b0001111; //7
■         4'h8: segdat_reg = 7'b0000000; //8
■         4'h9: segdat_reg = 7'b0000100; //9
■         default: segdat_reg = 7'b0000001;
■     endcase
■ end

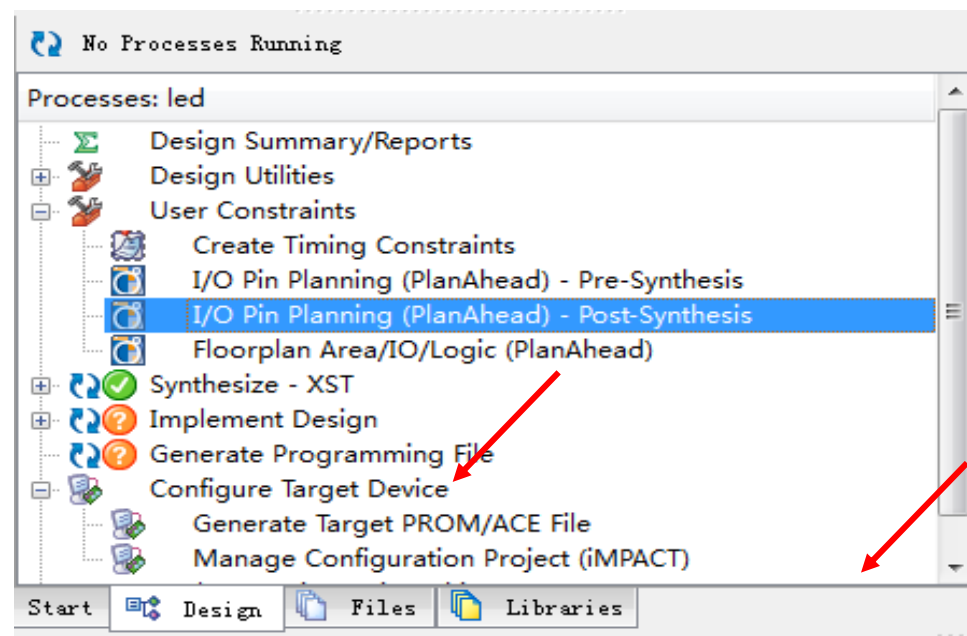
■ always @ (count[11:10])
■ begin
■     case (count[11:10])
■         2'b00: sl_reg = 4'b1110; //scan ge wei
■         2'b01: sl_reg = 4'b1101; //scan shi wei
■         2'b10: sl_reg = 4'b1011; //scan bai wei
■         2'b11: sl_reg = 4'b0111; //scan qian wei
■     endcase
■ end

```



流水灯---图形化界面设置管脚约束

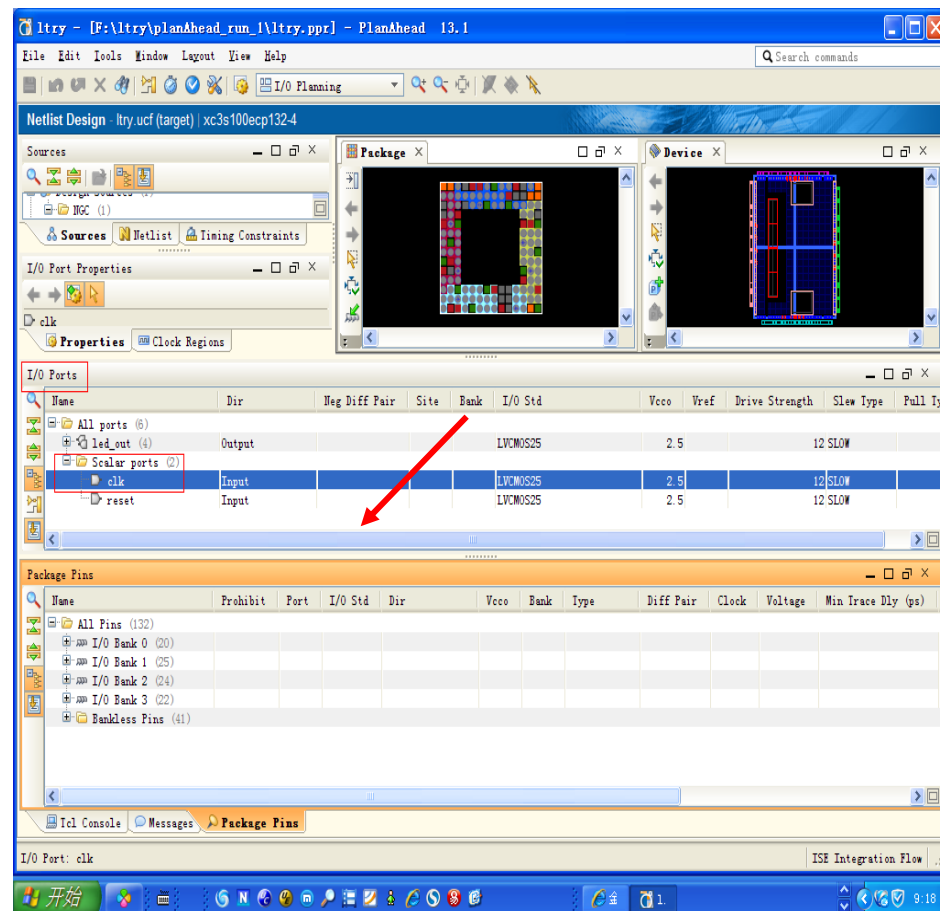
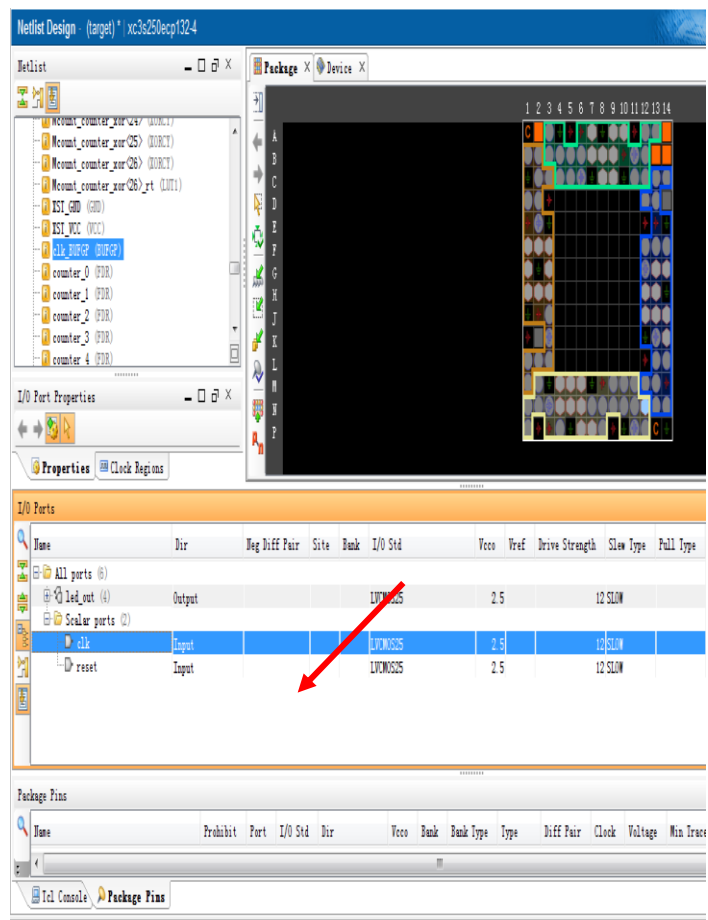
(5) 开始管脚约束：回到Process窗口选择User Constraints中的I/O Pin Planning (PlanAhead) - Post-Synthesis，双击打开PlanAhead界面



如果出现对话框问是否将所做修改保存到ucf文件，点击Yes

流水灯---图形化界面设置管脚约束

(6) PlanAhead中对管脚进行设定 找到I/O Ports窗口中的clk端口



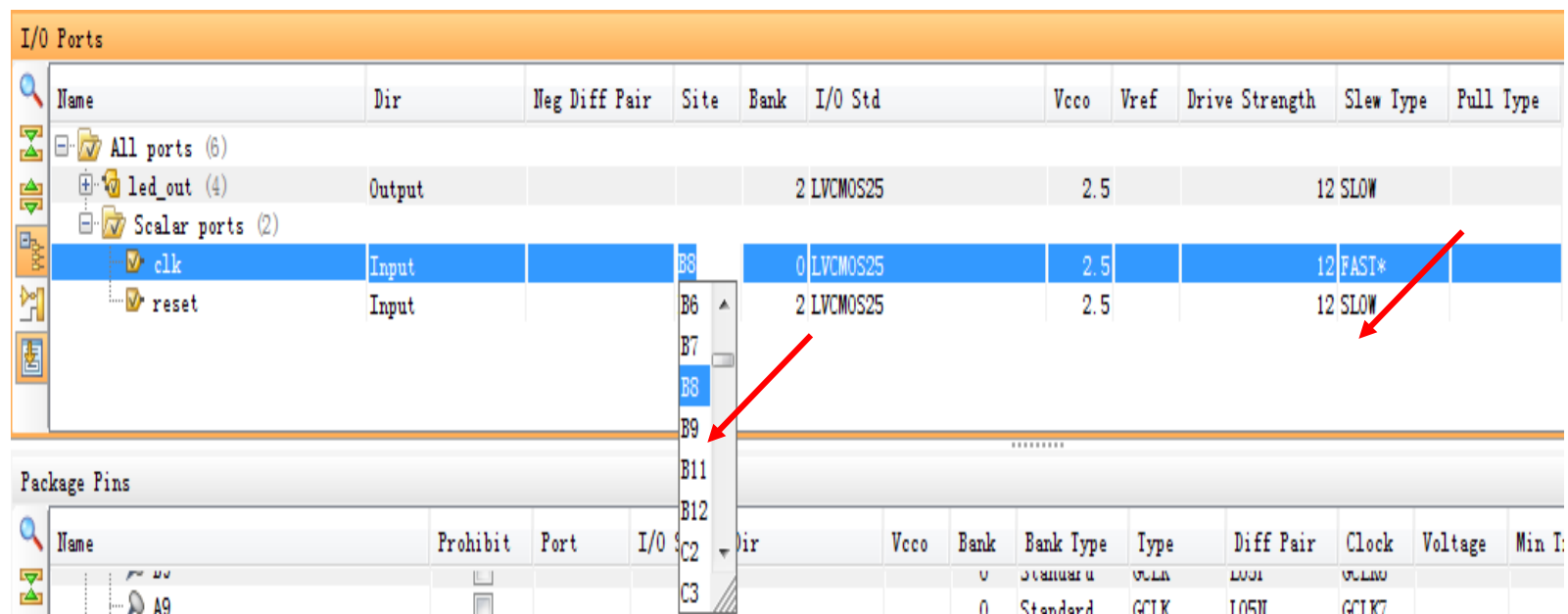
流水灯---图形化界面设置管脚约束

(7) 鼠标点在Site，在下拉窗口中选择B8

I/O Std: 电平标准 默认LVCMOS25

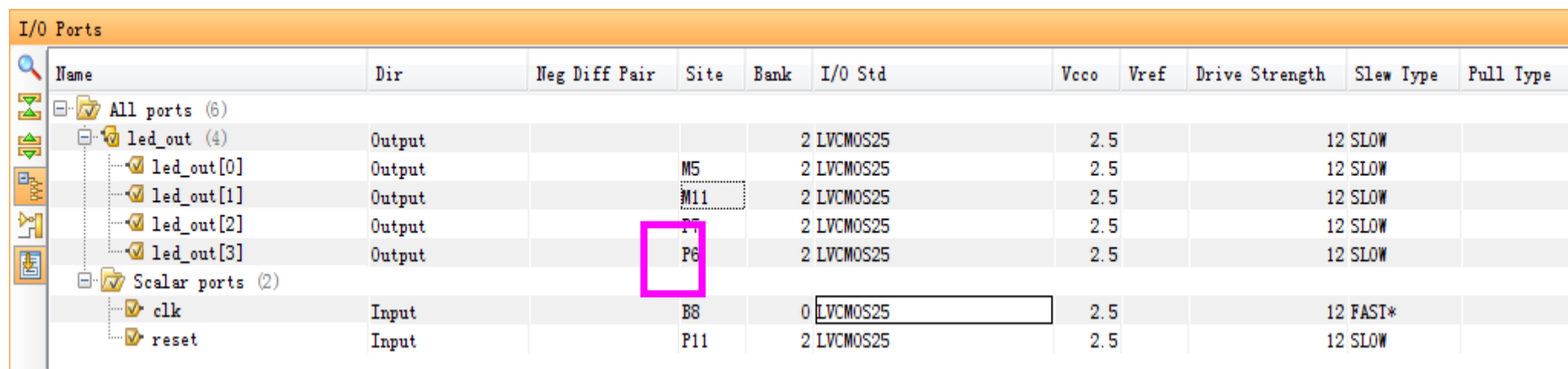
Drive Strength驱动强度: 默认12mA

Slew Type翻转速率: 默认Slow, 改为**FAST** (有的版本可能不能改)



流水灯---图形化界面设置管脚约束

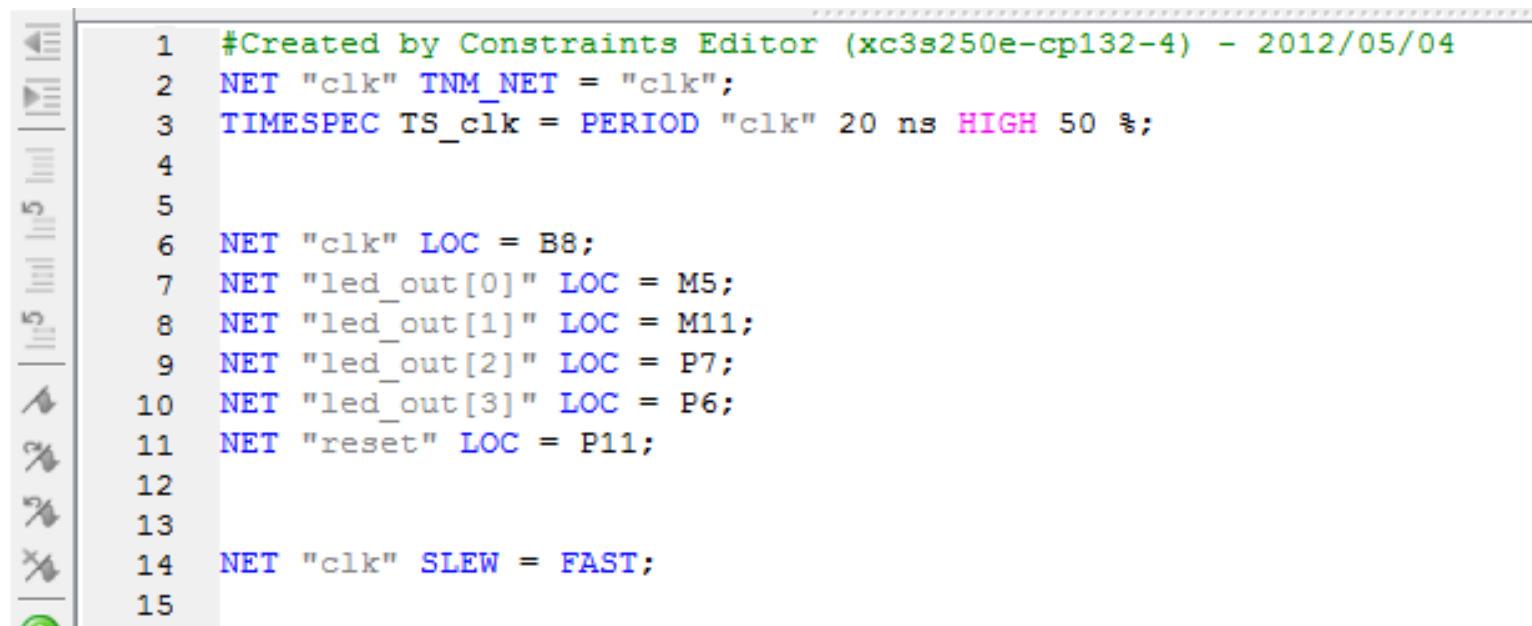
(8) 选择reset 为拨码开关SW0；其管脚位置为P11；
选择LED0~ LED3 为LD0~ LD3；其管脚位置分别为M5，M11，P6， P7



Name	Dir	Neg Diff Pair	Site	Bank	I/O Std	Vcco	Vref	Drive Strength	Slew Type	Pull Type
All ports (6)										
led_out (4)	Output				2 LVCMOS25	2.5		12 SLOW		
led_out[0]	Output		M5		2 LVCMOS25	2.5		12 SLOW		
led_out[1]	Output		M11		2 LVCMOS25	2.5		12 SLOW		
led_out[2]	Output		P7		2 LVCMOS25	2.5		12 SLOW		
led_out[3]	Output		P6		2 LVCMOS25	2.5		12 SLOW		
Scalar ports (2)										
clk	Input		B8		0 LVCMOS25	2.5		12 FAST*		
reset	Input		P11		2 LVCMOS25	2.5		12 SLOW		

流水灯---图形化界面设置管脚约束

- (9) 保存并退出PlanAhead。
- (10) 回到ISE界面，选Sources窗口内的led.ucf
点击processess窗口里的user constraints
双击edit constraints (txt)



```
1 #Created by Constraints Editor (xc3s250e-cp132-4) - 2012/05/04
2 NET "clk" TNM_NET = "clk";
3 TIMESPEC TS_clk = PERIOD "clk" 20 ns HIGH 50 %;
4
5
6 NET "clk" LOC = B8;
7 NET "led_out[0]" LOC = M5;
8 NET "led_out[1]" LOC = M11;
9 NET "led_out[2]" LOC = P7;
10 NET "led_out[3]" LOC = P6;
11 NET "reset" LOC = P11;
12
13
14 NET "clk" SLEW = FAST;
15
```

可以看到之前所做的约束都自动保存在UCF文件中。

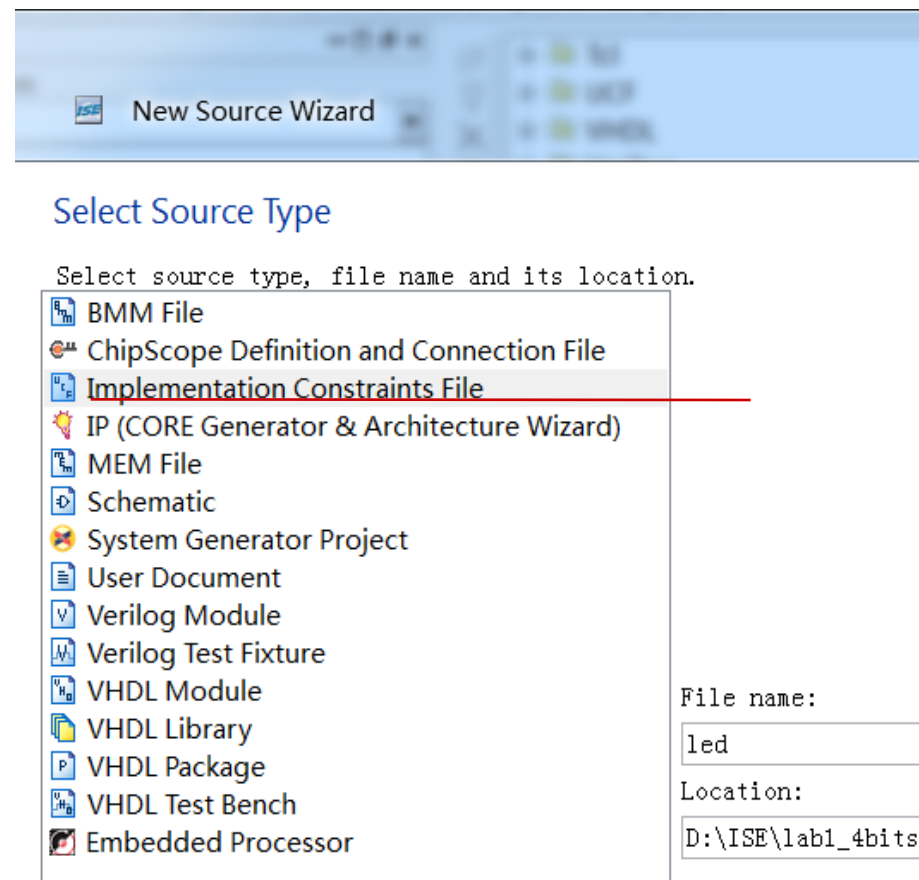
流水灯---直接编辑UCF文件创建和修改约束

管脚约束也可以直接编辑UCF 文件来进行。

(1) Project → New Source

**Implementation
Constraints File,**
文件名称为led(自动
储存为led.UCF)

(2) 一直点击**next**,
最后按**finish**



流水灯---直接编辑UCF文件创建和修改约束

- (3) 选择**source for Implementation**界面
- (4) 点选**Sources**窗口内的**led.ucf**，点击**processes**窗口里的**user constraints**，双击**edit constraints (txt)**，输入或修改其内容后储存文件。

- **LOC**: 管脚号
- **IOSTANDARD**: 电平标准
本实验设为LVCMOS33
- **“SLEW”**: 信号翻转速率
有fast和Slow(默认)

```
1 NET "clk" LOC = B8;  
2 NET "led_out[0]" LOC = M5;  
3 NET "led_out[1]" LOC = M11;  
4 NET "led_out[2]" LOC = P7;  
5 NET "led_out[3]" LOC = P6;  
6 NET "reset" LOC = P11;  
7  
8 NET "clk" IOSTANDARD = LVCMOS33;  
9 NET "led_out[0]" IOSTANDARD = LVCMOS33;  
10 NET "led_out[1]" IOSTANDARD = LVCMOS33;  
11 NET "led_out[2]" IOSTANDARD = LVCMOS33;  
12 NET "led_out[3]" IOSTANDARD = LVCMOS33;  
13 NET "reset" IOSTANDARD = LVCMOS33;  
14 NET "clk" SLEW = FAST;
```

数据类型

■ wire型

- 用**assign**赋值的组合逻辑信号，输入输出信号默认为**wire**型，取值为**0**、**1**、**x**和**z**

■ reg型

- 在**always**块内被赋值的任何信号都必须定义成**reg**型
- 注意：**reg**型并表示触发器的输出信号

■ memory型

- 通过**reg**型变量建立数组对存储器建模，如
- **reg[7:0] mem[255:0];**

Verilog语言描述方式来设计数字逻辑系统常见问题

模块逻辑功能

■ 有3种形式描述逻辑功能：

➤ **assign**语句—数据流方式描述

```
assign a=b&c;
```

➤ 用元件例化—结构化描述

```
and u1(c,a,b);
```

➤ 用**always**语句—行为方式描述

```
always @ ( b or c)
```

```
begin
```

```
    a = b&c;
```

```
end
```

Verilog语言描述方式来设计数字逻辑系统常见问题

1. 层次调用混乱

没有严格的按照Verilog语法来完成层次调用，需要学生学习Verilog语法中对于层次调用的三种正确的表达方式

2. 按照C语言函数描述的方式来描述逻辑电路

没有理解在C语言的函数描述中，变量或者语句的执行是有先后关系的，是“串行”方式，而verilog描述逻辑电路的状态变化是“并行”方式的。需要从认识上理解语句的描述对应到数字电路的电路特性

3. 阻塞赋值和非阻塞赋值的随意使用

一般情况下，阻塞赋值（用=表示）用来描述组合逻辑，非阻塞赋值（用<=表示）用来描述时序逻辑。经典的理解是：阻塞赋值表示用线来直接连接将结果传递给后续变量；非阻塞赋值是将结果经过了一个D触发器来完成，在下一个时钟的沿跳有效后才完成数据传递，保证数据的稳定性

赋值语句使用的原则

- 时序电路用非阻塞赋值；
- 组合逻辑时用阻塞赋值；
- 同时实现组合和时序逻辑时，使用非阻塞方式
- 当组合逻辑复杂度较高时，可将组合逻辑和时序分开实现
- 不要在同一个**always**块中，同时使用阻塞和非阻塞赋值；
- 不能在多个**always**块中，同时对一个变量进行赋值。

Verilog语言描述方式来设计数字逻辑系统常见问题

4. 异步逻辑和同步逻辑使用不当

异步逻辑使用不当会造成时序不稳定，如果尽可能使用同步逻辑电路完成设计

5. 多个always块中对同一个寄存器变量赋值

这种情况下一般会编译报错，此种情况的描述等同于将多个电路的输出直接线与了

在always块内被赋值的任何信号都必须定义成

reg型 注意：reg型表示触发器的输出信号

6. 使用不可综合的语法描述逻辑电路的“行为过程”

学生在有些时候使用verilog语法正确的但是不会被综合的方式来描述逻辑电路的行为过程，会导致最终的输出和设计者的想法不一致（如使用延迟描述#10来延迟一个信号的产生）